05장 파이썬 날개 달기

05장에서는 프로그래밍의 꽃이라 할 수 있는 클래스, 모듈, 예외 처리, 파이썬 라이브러리에 대해 알아보자. 이 장을 마치면 파이썬 프로그램을 작성하는 데 필요한 대부분의 내용을 배우게 된다.

목차

- 05-1 클래스
- 05-2 모듈
- 05-3 패키지
- 05-4 예외 처리
- 05-5 내장 함수
- 05-6 표준 라이브러리
- 05-7 외부 라이브러리
- 05장 되새김 문제

05-1 클래스

초보 개발자에게 클래스(class)는 넘기 힘든 장벽과도 같은 존재이다. 독자 중에도 클래스라는 단어를 처음 접하는 이들이 있을 것이다. 그러면 도대체 클래스가 무엇인지, 클래스가 왜 필요한지 매우 기초적인 것부터 차근차근 함께 알아보자.

- 클래스는 왜 필요한가?
    - 계산기 프로그램을 만들며 클래스 알아보기
- 클래스와 객체
- 사칙 연산 클래스 만들기
    - 클래스를 어떻게 만들지 먼저 구상하기
    - 클래스 구조 만들기
    - 객체에 연산할 숫자 지정하기
    - 더하기 기능 만들기
    - 곱하기, 빼기, 나누기 기능 만들기
- 생성자
- 클래스의 상속
- 메서드 오버라이딩
- 클래스변수

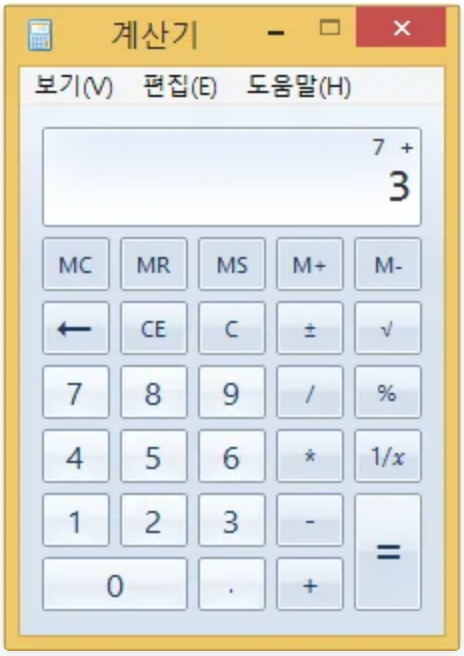
클래스는 왜 필요한가?

프로그래머들이 가장 많이 사용하는 프로그래밍 언어 중 하나인 C 언어에는 클래스가 없다. 이 말은 굳이 클래스가 없어도 프로그램을 충분히 만들 수 있다는 뜻이다. 파이썬으로 잘 만든 프로그램을 살펴봐도 클래스를 사용하지 않고 작성한 것이 매우 많다. 즉, 클래스는 지금까지 공부한 함수나 자료형처럼 프로그램 작성을 위해 꼭 필요한 요소는 아니다.

하지만 프로그램을 작성할 때 클래스를 적재적소에 사용하면 프로그래머가 얻을 수 있는 이익은 많다. 예제를 통해 한번 생각해 보자.

계산기 프로그램을 만들며 클래스 알아보기

여러분 모두 계산기를 사용해 보았을 것이다. 계산기에 숫자 3을 입력하고 +를 입력한 후 4를 입력하면 결괏값으로 7을 보여 준다. 다시 한번 +를 입력한 후 3을 입력하면 기존 결괏값 7에 3을 더해 10을 보여 준다. 즉, 계산기는 이전에 계산한 결괏값을 항상 메모리 어딘가에 저장하고 있어야 한다.



> *계산기는 이전에 계산한 결괏값을 기억하고 있어야 한다.*

이런 내용을 우리가 앞에서 익힌 함수를 이용해 구현해 보자. 계산기의 '더하기' 기능을 구현한 파이썬 코드는 다음과 같다.

```
# calculator.py
result = 0

def add(num):
    global result
    result += num  # 결괏값(result)에 입력값(num) 더하기
    return result  # 결괏값 반환

print(add(3))
print(add(4))
```

입력값을 이전에 계산한 결괏값에 더한 후 반환하는 add 함수를 위와 같이 작성했다. 이전에 계산한 결괏값을 유지하기 위해서 result 전역 변수(global)를 사용했다. 프로그램을 실행하면 예상한 대로 다음과 같은 결괏값이 출력된다.

```
3
7
```

그런데 만일 한 프로그램에서 2대의 계산기가 필요한 상황이 발생하면 어떻게 해야 할까? 각 계산기는 각각의 결괏값을 유지해야 하므로 위와 같이 add 함수 하나만으로는 결괏값을 따로 유지할 수 없다.

이런 상황을 해결하려면 다음과 같이 함수를 각각 따로 만들어야 한다.

```
# calculator2.py
result1 = 0
result2 = 0

def add1(num):  # 계산기1
    global result1
    result1 += num
    return result1

def add2(num):  # 계산기2
    global result2
    result2 += num
    return result2

print(add1(3))
print(add1(4))
print(add2(3))
print(add2(7))
```

똑같은 일을 하는 add1과 add2 함수를 만들고 각 함수에서 계산한 결괏값을 유지하면서 저장하는 전역 변수 result1과 result2를 정의했다.

결괏값은 다음과 같이 의도한 대로 출력된다.

```
3
7
3
10
```

계산기 1의 결괏값이 계산기 2에 아무런 영향을 끼치지 않음을 확인할 수 있다. 하지만 계산기가 3개, 5개, 10개로 점점 더 많이 필요해진다면 어떻게 해야 할까? 그때마다 전역 변수와 함수를 추가할 것인가? 여기에 계산기마다 빼기나 곱하기와 같은 기능을 추가해야 한다면 상황은 점점 더 복잡해질 것이다.

아직 클래스에 대해 배우지 않았지만, 위와 같은 경우에 클래스를 사용하면 다음과 같이 간단하게 해결할 수 있다.

> *다음 예시 클래스를 아직은 이해하지 못해도 좋다. 곧 자세하게 배울 것이다. 여기에서는 클래스 개념만 어렴풋이 이해하면 된다.*

```
# calculator3.py
class Calculator:
    def __init__(self):
        self.result = 0

    def add(self, num):
        self.result += num
        return self.result

cal1 = Calculator()
cal2 = Calculator()

print(cal1.add(3))
print(cal1.add(4))
print(cal2.add(3))
print(cal2.add(7))
```

프로그램을 실행하면 함수 2개를 사용했을 때와 동일한 결과가 출력된다.

```
3
7
3
10
```

Calculator 클래스로 만든 별개의 계산기 cal1, cal2(파이썬에서는 이것을 '객체'라고 부른다)가 각각의 역할을 수행한다. 그리고 계산기의 결괏값 역시 다른 계산기의 결괏값과 상관없이 독립적인 값을 유지한다. 이렇게 클래스를 사용하면 계산기 대수가 늘어나도 객체를 생성하면 되므로 함수만 사용할 때보다 간단하게 프로그램을 작성할 수 있다. 빼기 기능을 더하고 싶다면 Calculator 클래스에 다음과 같이 빼기 기능을 가진 함수를 추가하면 된다.

```
class Calculator:
    def __init__(self):
        self.result = 0

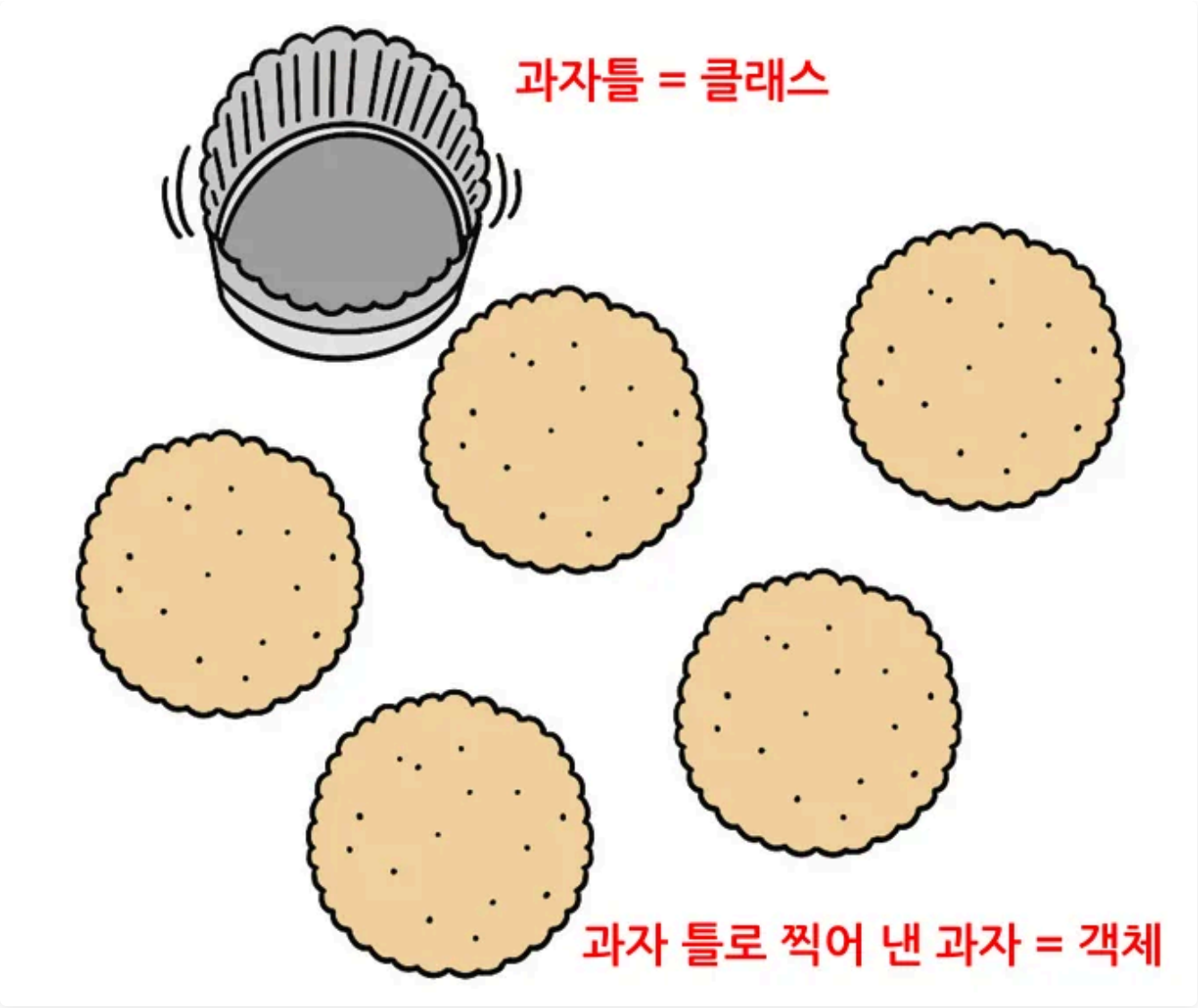
    def add(self, num):
        self.result += num
        return self.result

    [[MARK]]def sub(self, num):
        self.result -= num
        return self.result[[/MARK]]
```

클래스의 이점은 이것만이 아니다. 하지만 이 예시 하나만으로도 '도대체 왜 클래스가 필요한 것일까?'라는 근본적인 물음에 답이 되었을 것이다.

클래스와 객체

클래스를 가장 잘 설명해 주는 다음 그림을 살펴보자. 과자를 만드는 과자 틀과 이를 사용해 만든 과자이다.



- 과자 틀 = 클래스
- 과자 틀로 찍어 낸 과자 = 객체

여기에서 설명할 클래스는 과자 틀과 비슷하다. 클래스(class)란 똑같은 무언가를 계속 만들어 낼 수 있는 설계 도면(과자 틀)이고, 객체(object)란 클래스로 만든 피조물(과자 틀로 찍어 낸 과자)을 뜻한다.

클래스로 만든 객체에는 중요한 특징이 있다. 바로 객체마다 고유한 성격을 가진다는 것이다. 과자 틀로 만든 과자에 구멍을 뚫거나 조금 베어 먹더라도 다른 과자에는 아무런 영향이 없는 것과 마찬가지로 동일한 클래스로 만든 객체들은 서로 전혀 영향을 주지 않는다.

다음은 파이썬 클래스의 가장 간단한 예이다.

```
>>> class Cookie:
...     pass
...
>>>
```

위에서 작성한 Cookie 클래스는 아무런 기능도 가지고 있지 않은 껍질뿐인 클래스이다. 하지만 이렇게 껍질뿐인 클래스도 객체를 생성하는 기능이 있다. '과자 틀'로 '과자'를 만드는 것처럼 말이다.

객체는 클래스로 만들고 1개의 클래스는 무수히 많은 객체를 만들어 낼 수 있다. 위에서 만든 Cookie 클래스의 객체를 만드는 방법은 다음과 같다.

```
>>> a = Cookie()
>>> b = Cookie()
```

Cookie()의 결괏값을 반환받은 a와 b가 바로 객체이다. 마치 함수를 사용해서 그 결괏값을 반환받는 모습과 비슷하다.

점프 투 파이썬

객체와 인스턴스의 차이

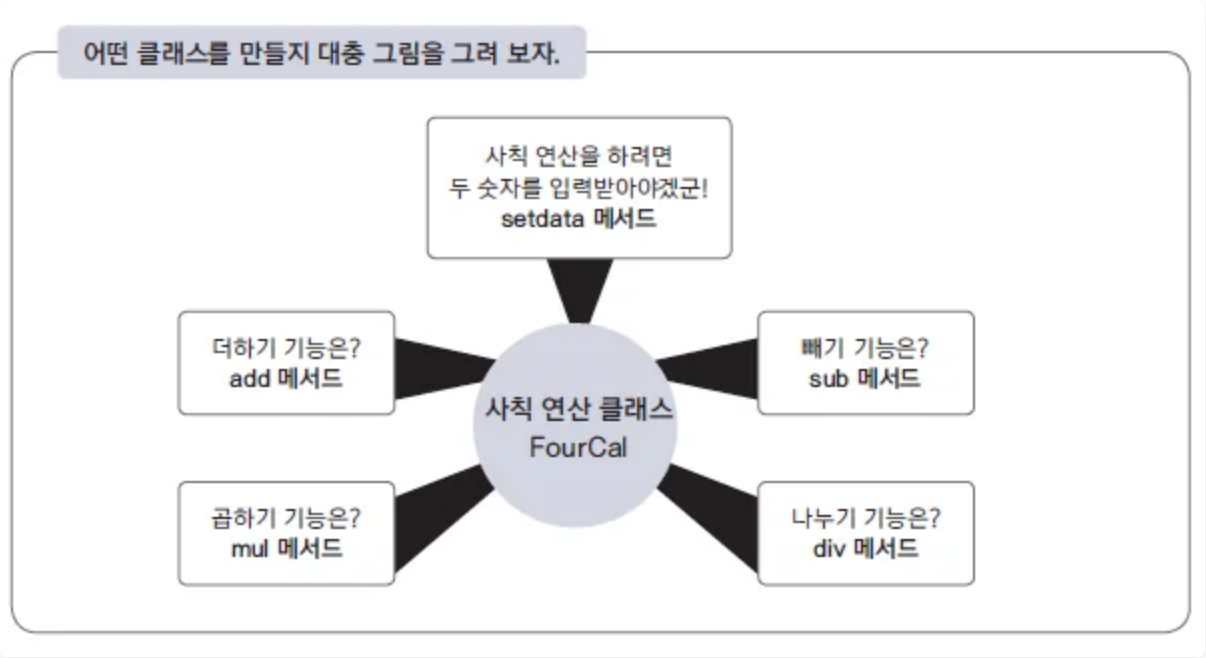
클래스로 만든 객체를 '인스턴스'라고도 한다. 그렇다면 객체와 인스턴스의 차이는 무엇일까? 이렇게 생각해 보자. `a = Cookie()`로 만든 a는 객체이다. 그리고 a 객체는 Cookie의 인스턴스이다. 즉, 인스턴스라는 말은 특정 객체(a)가 어떤 클래스(Cookie)의 객체인지를 관계 위주로 설명할 때 사용한다. 'a는 인스턴스'보다 'a는 객체'라는 표현이 어울리며 'a는 Cookie의 객체'보다 'a는 Cookie의 인스턴스'라는 표현이 훨씬 잘 어울린다.

사칙 연산 클래스 만들기

'백견(百見)이 불여일타(一打)'라고 했다. 클래스를 직접 만들며 배워 보자. 여기에서는 사칙 연산을 하는 클래스를 만들어 볼 것이다. 사칙 연산은 더하기, 빼기, 곱하기, 나누기를 말한다.

클래스를 어떻게 만들지 먼저 구상하기

클래스는 무작정 만드는 것보다 클래스로 만든 객체를 중심으로 어떤 식으로 동작하게 할지 미리 구상한 후 생각한 것을 하나씩 만들면서 완성해 나가는 것이 좋다.



사칙 연산 기능을 가진 FourCal 클래스가 다음처럼 동작한다고 가정해 보자.

먼저 a = FourCal()를 입력해서 a라는 객체를 만든다.

```
>>> a = FourCal()
```

그런 다음 `a.setdata(4, 2)` 처럼 입력해서 숫자 4와 2를 a에 지정해 준다.

```
>>> a.setdata(4, 2)
```

a.add()를 수행하면 두 수를 합한 결과(`4 + 2`)를 반환한다.

```
>>> a.add()
6
```

a.mul()을 수행하면 두 수를 곱한 결과(`4 * 2`)를 반환한다.

```
>>> a.mul()
8
```

a.sub()를 수행하면 두 수를 뺀 결과(`4 - 2`)를 반환한다.

```
>>> a.sub()
2
```

a.div()를 수행하면 두 수를 나눈 결과(`4 / 2`)를 반환한다.

```
>>> a.div()
2.0
```

이렇게 동작하는 FourCal 클래스를 만드는 것이 바로 우리의 목표이다.

클래스 구조 만들기

그러면 지금부터 앞에서 구상한 것처럼 동작하는 클래스를 만들어 보자. 제일 먼저 할 일은 `a = FourCal()` 처럼 객체를 만들 수 있게 하는 것이다. 일단은 아무런 기능이 없어도 되기 때문에 매우 간단하게 만들 수 있다. 다음을 따라 해 보자.

```
>>> class FourCal:
...     pass
```

먼저 대화형 인터프리터에서 pass라는 문장만을 포함한 FourCal 클래스를 만든다. 현재 상태에서 FourCal 클래스는 아무 변수나 함수도 포함하지 않지만, 우리가 원하는 객체 a를 만들 수 있는 기능은 가지고 있다. 한번 확인해 보자.

> *pass는 아무것도 수행하지 않는 문법으로, 임시로 코드를 작성할 때 주로 사용한다.*

```
>>> a = FourCal()
>>> type(a)
<class '__main__.FourCal'>
```

위와 같이 `a = FourCal()` 로 a 객체를 먼저 만든 후 type(a)로 a 객체가 어떤 타입인지 알아보았다. 역시 객체 a가 FourCal 클래스의 인스턴스임을 알 수 있다.

> *type은 파이썬이 자체로 가지고 있는 내장 함수로, 객체의 타입을 출력한다.*

객체에 연산할 숫자 지정하기

하지만 생성된 객체 a는 아직 아무런 기능도 하지 못한다. 이제 더하기, 빼기, 곱하기, 나누기 등의 기능을 하는 객체를 만들어야 한다. 그런데 이러한 기능을 갖춘 객체를 만들려면 먼저 사칙 연산을 할 때 사용할 2개의 숫자를 a 객체에게 알려 주어야 한다. 다음과 같이 연산을 수행할 대상(4, 2)을 객체에 지정할 수 있게 만들어 보자.

```
>>> a.setdata(4, 2)
```

위 문장이 동작하려면 다음과 같이 FourCal 클래스를 다시 정의해야 한다.

```
>>> class FourCal:
...     def setdata(self, first, second):
...         self.first = first
...         self.second = second
```

앞에서 만든 FourCal 클래스에서 pass 문장을 삭제하고 그 대신 setdata 함수를 정의했다. 클래스 안에 구현된 함수는 다른 말로 **메서드(method)**라고 부른다. 앞으로 클래스 내부의 함수는 항상 메서드라고 표현할 테니 메서드라는 용어를 기억해 두자.

일반적인 함수를 만들 때는 다음과 같이 소스 코드를 작성한다.

```
def 함수_이름(매개변수):
    수행할_문장
    ...
```

메서드도 클래스에 포함되어 있다는 점만 제외하면 일반 함수와 다를 것이 없다.

setdata 메서드를 다시 보면 다음과 같다.

```
def setdata(self, first, second):    # 메서드의 매개변수
    self.first = first               # 메서드의 수행문
    self.second = second             # 메서드의 수행문
```

setdata 메서드의 매개변수

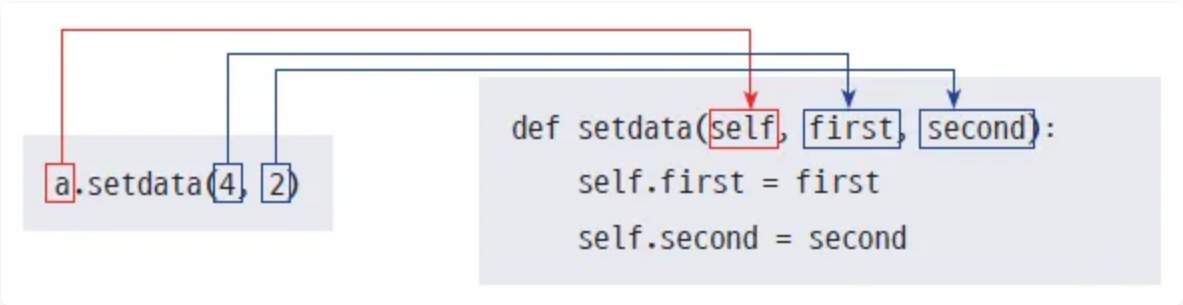
setdata 메서드를 좀 더 자세히 살펴보자. setdata 메서드는 매개변수로 self, first, second 3개의 입력값을 받는다. 그런데 일반 함수와 달리, 메서드의 첫 번째 매개변수 self는 특별한 의미를 가진다.

다음과 같이 a 객체를 만들고 a 객체를 통해 setdata 메서드를 호출해 보자.

```
>>> a = FourCal()
>>> a.setdata(4, 2)
```

> *객체를 이용해 클래스의 메서드를 호출하려면 a.setdata(4, 2)와 같이 도트(.) 연산자를 사용하면 된다.*

그런데 뭔가 좀 이상하지 않은가? setdata 메서드에는 self, first, second 총 3개의 매개변수가 필요한데 실제로는 `a.setdata(4, 2)` 처럼 2개의 값만 전달했다. 왜 그럴까? `a.setdata(4, 2)` 처럼 호출하면 setdata 메서드의 첫 번째 매개변수 self에는 setdata 메서드를 호출한 객체 a가 자동으로 전달되기 때문이다. 다음 그림을 보면 객체의 메서드를 호출할 때 입력한 값이 메서드에 어떻게 전달되는지 쉽게 이해할 수 있을 것이다.



파이썬 메서드의 첫 번째 매개변수 이름은 관례적으로 self를 사용한다. 객체의 메서드를 호출할 때 호출한 객체 자신이 전달되기 때문에 self라는 이름을 사용한 것이다. 물론 self 말고 다른 이름을 사용해도 상관없다.

> *메서드의 첫 번째 매개변수 self를 명시적으로 구현하는 것은 파이썬만의 독특한 특징이다. 예를 들어 자바와 같은 언어는 첫 번째 매개변수 self가 필요 없다.*

점프 투 파이썬

메서드를 호출하는 또 다른 방법

잘 사용하지는 않지만, 다음과 같이 클래스를 이용해 메서드를 호출할 수도 있다.

```
>>> a = FourCal()
>>> FourCal.setdata(a, 4, 2)
```

위와 같이 '클래스명.메서드' 형태로 호출할 때는 객체 a를 첫 번째 매개변수 self에 꼭 전달해야 한다. 반면 다음처럼 '객체.메서드' 형태로 호출할 때는 self를 반드시 생략해서 호출해야 한다.

```
>>> a = FourCal()
>>> a.setdata(4, 2)
```

setdata 메서드의 수행문

이제 setdata 메서드의 수행문에 대해 알아보자.

```
def setdata(self, first, second):    # 메서드의 매개변수
    self.first = first               # 메서드의 수행문
    self.second = second             # 메서드의 수행문
```

`a.setdata(4, 2)` 처럼 호출하면 setdata 메서드의 매개변수 first, second에는 각각 값 4와 2가 전달되어 setdata 메서드의 수행문이 다음과 같이 해석된다.

```
self.first = 4
self.second = 2
```

self는 전달된 객체 a이므로 다시 다음과 같이 해석된다.

```
a.first = 4
a.second = 2
```

`a.first = 4` 라는 문장이 수행되면 a 객체에 객체변수 first가 생성되고 4라는 값이 저장된다. 마찬가지로 `a.second = 2` 라는 문장이 수행되면 a 객체에 객체변수 second가 생성되고 2라는 값이 저장된다.

> *객체에 생성되는 객체변수를 '인스턴스 변수' 또는 '속성'이라고도 부른다.*

다음과 같이 확인해 보자.

```
>>> a = FourCal()
>>> a.setdata(4, 2)
>>> a.first
4
>>> a.second
2
```

a 객체에 객체변수 first와 second가 생성된 것을 확인할 수 있다.

이번에는 다음과 같이 a, b 객체를 만들어 보자.

```
>>> a = FourCal()
>>> b = FourCal()
```

그리고 a 객체의 객체변수 first를 다음과 같이 생성한다.

```
>>> a.setdata(4, 2)
>>> a.first
4
```

이번에는 b 객체의 객체변수 first를 다음과 같이 생성한다.

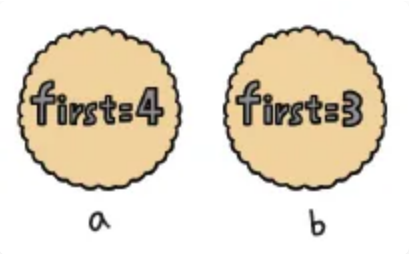
```
>>> b.setdata(3, 7)
>>> b.first
3
```

자, 이제 매우 중요한 질문을 하겠다. 위와 같이 진행하면 b 객체의 객체변수 first에는 값 3이 저장된다는 것을 확인할 수 있었다. 그렇다면 a 객체의 first에 저장된 값도 3으로 변할까, 아니면 원래대로 값 4를 유지할까?

다음과 같이 그 결과를 확인해 보자.

```
>>> a.first
4
```

a 객체의 first 값은 b 객체의 first 값에 영향받지 않고 원래 값을 유지하고 있음을 확인할 수 있다. 이 예제를 통해 강조하고 싶은 점이 바로 이것이다. 클래스로 만든 객체의 객체변수는 다른 객체의 객체변수에 상관없이 독립적인 값을 유지한다. 클래스에서는 이 부분을 이해하는 것이 가장 중요하다.



다음은 현재까지 완성된 FourCal 클래스이다.

```
>>> class FourCal:
...     def setdata(self, first, second):
...         self.first = first
...         self.second = second
```

지금까지 살펴본 내용이 바로 이 4줄의 소스 코드를 설명하기 위한 것이었다. 앞에서 설명한 것들이 이해되지 않는다면 다시 한 번 읽어 보기 바란다. 이 부분을 이해하지 못하면 다음으로 넘어갈 수 없기 때문이다.

더하기 기능 만들기

자! 2개의 숫자 값을 설정해 주었으니 2개의 숫자를 더하는 기능을 방금 만든 클래스에 추가해 보자. 우리는 다음과 같이 더하기 기능을 갖춘 클래스를 만들어야 한다.

```
>>> a = FourCal()
>>> a.setdata(4, 2)
>>> a.add()
6
```

이 연산이 가능하도록 FourCal 클래스를 다시 작성해 보자.

```
>>> class FourCal:
...     def setdata(self, first, second):
...         self.first = first
...         self.second = second
...     def add(self):
...         result = self.first + self.second
...         return result
```

> *05장에서는 대화형 인터프리터에서 FourCal 클래스를 계속 수정할 것이다. 그러나 클래스를 수정하는 데 있어 코드가 많아 조금 불편할 수 있다. 이 불편함은 IDLE 에디터를 사용해 FourCal 클래스를 작성하고 IDLE 에디터에서 작성한 코드를 복사한 후 대화형 인터프리터에 붙여넣기하여 해결할 수 있다.*

add 메서드를 새롭게 추가했다. 이제 클래스를 사용해 보자.

```
>>> a = FourCal()
>>> a.setdata(4, 2)
```

위와 같이 호출하면 앞에서 살펴보았듯이 a 객체의 first, second 객체변수에는 각각 값 4와 2가 저장될 것이다.

이제 add 메서드를 호출해 보자.

```
>>> a.add()
6
```

a.add()라고 호출하면 add 메서드가 호출되어 값 6이 출력될 것이다. 어떤 과정을 거쳐 값 6이 출력되는지 add 메서드를 따로 떼어 내 자세히 살펴보자.

```
def add(self):
    result = self.first + self.second
    return result
```

add 메서드의 매개변수는 self, 반환값은 result이다. 반환값인 result를 계산하는 부분은 다음과 같다.

```
result = self.first + self.second
```

a.add()와 같이 a 객체에 의해 add 메서드가 수행되면 add 메서드의 self에는 객체 a가 자동으로 입력되므로 이 내용은 다음과 같이 해석된다.

```
result = a.first + a.second
```

a.first와 a.second는 add 메서드가 호출되기 전에 `a.setdata(4, 2)` 문장에서 `a.first = 4`, `a.second = 2`로 설정된다. 따라서 위 문장은 다시 다음과 같이 해석된다.

```
result = 4 + 2
```

따라서 다음과 같이 a.add()를 호출하면 6을 반환한다.

```
>>> a.add()
6
```

여기까지 모두 이해한 독자라면 클래스에 대해 80% 이상을 안 것이다. 파이썬의 클래스는 그다지 어렵지 않다.

곱하기, 빼기, 나누기 기능 만들기

이번에는 곱하기, 빼기, 나누기 등을 할 수 있도록 프로그램을 개선해 보자.

```
>>> class FourCal:
...     def setdata(self, first, second):
...         self.first = first
...         self.second = second
...     def add(self):
...         result = self.first + self.second
...         return result
[[MARK]]...     def mul(self):
...         result = self.first * self.second
...         return result
...     def sub(self):
...         result = self.first - self.second
...         return result
...     def div(self):
...         result = self.first / self.second
...         return result[[/MARK]]
```

mul, sub, div 모두 add 메서드에서 배운 것과 동일한 방법이므로 따로 설명하지는 않겠다.

정말 모든 것이 제대로 동작하는지 확인해 보자.

```
>>> a = FourCal()
>>> b = FourCal()
>>> a.setdata(4, 2)
>>> b.setdata(3, 8)
>>> a.add()
6
>>> a.mul()
8
>>> a.sub()
2
>>> a.div()
2.0
>>> b.add()
11
>>> b.mul()
24
>>> b.sub()
-5
>>> b.div()
0.375
```

지금까지 우리가 목표로 한 사칙 연산 기능을 가진 클래스를 만들어 보았다.

생성자

이번에는 우리가 만든 FourCal 클래스를 다음과 같이 사용해 보자.

```
>>> a = FourCal()
>>> a.add()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 6, in add
AttributeError: 'FourCal' object has no attribute 'first'
```

FourCal 클래스의 인스턴스 a에 setdata 메서드를 수행하지 않고 add 메서드를 먼저 수행하면 'AttributeError: 'FourCal' object has no attribute 'first'' 오류가 발생한다. setdata 메서드를 수행해야 객체 a의 객체변수 first와 second가 생성되기 때문이다. 이렇게 객체에 first, second와 같은 초깃값을 설정해야 할 필요가 있을 때는 setdata와 같은 메서드를 호출하여 초깃값을 설정하기보다 생성자를 구현하는 것이 안전한 방법이다.

생성자(constructor)란 객체가 생성될 때 자동으로 호출되는 메서드를 의미한다. 파이썬 메서드명으로 `__init__`를 사용하면 이 메서드는 생성자가 된다.

다음과 같이 FourCal 클래스에 생성자를 추가해 보자.

> *`__init__` 메서드의 init 앞뒤로 붙은 `__` 는 밑줄(_) 2개를 붙여 쓴 것이다.*

```
>>> class FourCal:
...     [[MARK]]def __init__(self, first, second):
...         self.first = first
...         self.second = second[[/MARK]]
...     def setdata(self, first, second):
...         self.first = first
...         self.second = second
...     def add(self):
...         result = self.first + self.second
...         return result
...     def mul(self):
...         result = self.first * self.second
...         return result
...     def sub(self):
...         result = self.first - self.second
...         return result
...     def div(self):
...         result = self.first / self.second
...         return result
...
>>>
```

새롭게 추가된 생성자 `__init__` 메서드만 따로 떼어 내서 살펴보자.

```
def __init__(self, first, second):
    self.first = first
    self.second = second
```

`__init__` 메서드는 setdata 메서드와 이름만 다르고 모든 게 동일하다. 단, 메서드 이름을 `__init__`로 했기 때문에 생성자로 인식되어 객체가 생성되는 시점에 자동으로 호출된다는 차이가 있다.

이제 다음처럼 a 객체를 생성해 보자.

```
>>> a = FourCal()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: __init__() missing 2 required positional arguments: 'first' and 'second'
```

`a = FourCal()`을 수행할 때 생성자 `__init__`가 호출되어 위와 같은 오류가 발생했다. 오류가 발생한 이유는 생성자의 매개변수 first와 second에 해당하는 값이 전달되지 않았기 때문이다.

이 오류를 해결하려면 다음처럼 first와 second에 해당하는 값을 전달하여 객체를 생성해야 한다.

```
>>> a = FourCal(4, 2)
>>>
```

위와 같이 수행하면 `__init__` 메서드의 매개변수에는 각각 다음과 같은 값이 전달된다.

매개변수	값
self	생성되는 객체
first	4
second	2

> *`__init__` 메서드도 다른 메서드와 마찬가지로 첫 번째 매개변수 self에 생성되는 객체가 자동으로 전달된다는 점을 기억하자.*

따라서 `__init__` 메서드가 호출되면 setdata 메서드를 호출했을 때와 마찬가지로 first와 second라는 객체변수가 생성될 것이다.

다음과 같이 객체변수의 값을 확인해 보자.

```
>>> a = FourCal(4, 2)
>>> a.first
4
>>> a.second
2
```

add나 div 등과 같은 메서드도 잘 동작하는지 확인해 보자.

```
>>> a = FourCal(4, 2)
>>> a.add()
6
>>> a.div()
2.0
```

이상 없이 잘 동작하는 것을 확인할 수 있다.

클래스의 상속

상속(Inheritance)이란 '물려받다'라는 뜻으로, '재산을 상속받다'라고 할 때의 상속과 같은 의미이다. 클래스에도 이 개념을 적용할 수 있다. 어떤 클래스를 만들 때 다른 클래스의 기능을 물려받을 수 있게 만드는 것이다. 이번에는 상속 개념을 사용하여 우리가 만든 FourCal 클래스에 a^b 값을 구할 수 있는 기능을 추가해 보자.

앞에서 FourCal 클래스는 이미 만들어 놓았으므로 FourCal 클래스를 상속하는 MoreFourCal 클래스는 다음과 같이 간단하게 만들 수 있다.

```
>>> class MoreFourCal(FourCal):
...     pass
```

클래스를 상속하기 위해서는 다음처럼 클래스 이름 뒤 괄호 안에 상속할 클래스 이름을 넣어주면 된다.

```
class 클래스_이름(상속할_클래스_이름)
```

MoreFourCal 클래스는 FourCal 클래스를 상속했으므로 FourCal 클래스의 모든 기능을 사용할 수 있다.

다음과 같이 확인해 보자.

```
>>> a = MoreFourCal(4, 2)
>>> a.add()
6
>>> a.mul()
8
>>> a.sub()
2
>>> a.div()
2.0
```

상속받은 FourCal 클래스의 기능을 모두 사용할 수 있음을 확인할 수 있다.

점프 투 파이썬

상속 기능은 왜 쓰는 걸까?

보통 상속은 기존 클래스를 변경하지 않고 기능을 추가하거나 기존 기능을 변경하려고 할 때 사용한다. '클래스에 기능을 추가하고 싶으면 기존 클래스를 수정하면 되는데 왜 굳이 상속을 받아서 처리해야 하지?'라는 의문이 들 수도 있다. 하지만 기존 클래스가 라이브러리 형태로 제공되거나 수정이 허용되지 않는 상황이라면 상속을 사용해야 한다.

이제 원래 목적인 a^b을 계산하는 MoreFourCal 클래스를 만들어 보자.

```
>>> class MoreFourCal(FourCal):
...     def pow(self):
...         result = self.first ** self.second
...         return result
```

pass 문장은 삭제하고 위와 같이 두 수의 거듭제곱을 구할 수 있는 pow 메서드를 추가했다. 그리고 다음과 같이 pow 메서드를 수행해 보자.

```
>>> a = MoreFourCal(4, 2)
>>> a.pow()
16
>>> a.add()
6
```

MoreFourCal 클래스로 만든 a 객체에 값 4와 2를 지정한 후 pow 메서드를 호출하면 4의 2제곱(4^2)인 16을 반환하는 것을 확인할 수 있다. 상속받은 기능인 add 메서드도 잘 동작한다.

상속은 MoreFourCal 클래스처럼 기존 클래스(FourCal)는 그대로 놔둔 채 클래스의 기능을 확장할 때 주로 사용한다.

메서드 오버라이딩

이번에는 FourCal 클래스를 다음과 같이 실행해 보자.

```
>>> a = FourCal(4, 0)
>>> a.div()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    result = self.first / self.second
ZeroDivisionError: division by zero
```

FourCal 클래스의 객체 a에 값 4와 0을 지정하고 div 메서드를 호출하면 4를 0으로 나누려고 하므로 ZeroDivisionError 오류가 발생한다. 0으로 나눌 때 오류가 아닌 값 0을 반환받고 싶다면 어떻게 해야 할까?

다음과 같이 FourCal 클래스를 상속하는 SafeFourCal 클래스를 만들어 보자.

```
>>> class SafeFourCal(FourCal):
...     def div(self):
...         if self.second == 0:  # 나누는 값이 0인 경우 0을 반환하도록 수정
...             return 0
...         else:
...             return self.first / self.second
```

FourCal 클래스에 있는 div 메서드를 동일한 이름으로 다시 작성했다. 이렇게 부모 클래스(상속한 클래스)에 있는 메서드를 동일한 이름으로 다시 만드는 것을 **메서드 오버라이딩**(method overriding)이라고 한다. 이렇게 메서드를 오버라이딩하면 부모 클래스의 메서드 대신 오버라이딩한 메서드가 호출된다.

SafeFourCal 클래스에 오버라이딩한 div 메서드는 나누는 값이 0인 경우에는 0을 반환하도록 수정했다. 이제 다시 앞에서 수행한 예제를 FourCal 클래스 대신 SafeFourCal 클래스를 사용하여 수행해 보자.

```
>>> a = SafeFourCal(4, 0)
>>> a.div()
0
```

FourCal 클래스와 달리 ZeroDivisionError가 발생하지 않고 의도한 대로 0이 반환되는 것을 확인할 수 있다.

클래스변수

객체변수는 다른 객체들의 영향을 받지 않고 독립적으로 그 값을 유지한다는 점을 이미 알아보았다. 이번에는 객체변수와는 성격이 다른 클래스변수에 대해 알아보자.

다음 클래스를 작성해 보자.

```
>>> class Family:
...     lastname = "김"
```

Family 클래스에 선언한 lastname이 바로 클래스변수이다. 클래스변수는 클래스 안에 함수를 선언하는 것과 마찬가지로 클래스 안에 변수를 선언하여 생성한다.

이제 Family 클래스를 다음과 같이 사용해 보자.

```
>>> Family.lastname
김
```

클래스변수는 위 예와 같이 `클래스_이름.클래스변수` 로 사용할 수 있다.

또는 다음과 같이 Family 클래스로 만든 객체를 이용해도 클래스변수를 사용할 수 있다.

```
>>> a = Family()
>>> b = Family()
>>> a.lastname
김
>>> b.lastname
김
```

만약 Family 클래스의 lastname을 "박"이라는 문자열로 바꾸면 어떻게 될까? 다음과 같이 확인해 보자.

```
>>> Family.lastname = "박"
>>> a.lastname
박
>>> b.lastname
박
```

클래스변수의 값을 변경했더니 클래스로 만든 객체의 lastname 값도 모두 변경됨을 확인할 수 있다. 즉, 클래스변수는 객체변수와 달리 클래스로 만든 모든 객체에 공유된다는 특징이 있다.

클래스변수를 가장 늦게 설명하는 이유는 클래스에서 객체변수가 클래스변수보다 훨씬 중요하기 때문이다. 실무에서 프로그래밍할 때도 클래스변수보다 객체변수를 사용하는 비율이 훨씬 높다.

점프 투 파이썬

클래스변수와 동일한 이름의 객체변수를 생성하면?

위의 예제에서 a.lastname을 다음처럼 변경하면 어떻게 될까?

```
>>> a.lastname = "최"
>>> a.lastname
최
```

이렇게 하면 Family 클래스의 lastname이 바뀌는 것이 아니라 a 객체에 lastname이라는 객체변수가 새롭게 생성된다. 즉, 객체변수는 클래스변수와 동일한 이름으로 생성할 수 있다.

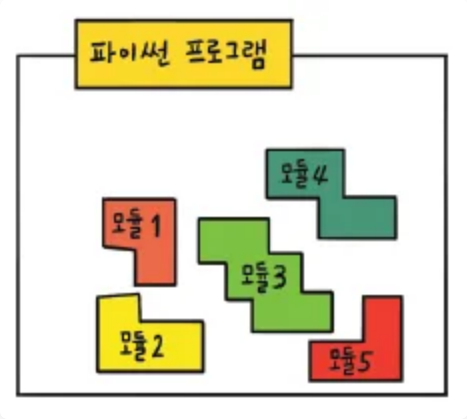
a.lastname 객체변수를 생성하더라도 Family 클래스의 lastname과는 상관없다는 것을 다음과 같이 확인할 수 있다.

```
>>> Family.lastname
박
>>> b.lastname
박
```

Family 클래스의 lastname 값은 변하지 않았다.

05-2 모듈

모듈이란 함수나 변수 또는 클래스를 모아 놓은 파이썬 파일이다. 다른 파이썬 프로그램에서 불러와 사용할 수 있게 만든 파이썬 파일이라고도 할 수 있다. 파이썬으로 프로그래밍을 할 때 매우 많은 모듈을 사용하는데, 다른 사람들이 이미 만들어 놓은 모듈을 사용할 수도 있고 직접 만들어 사용할 수도 있다. 여기에서는 모듈을 어떻게 만들고 사용하는지 알아본다.



- 모듈 만들기
- 모듈 불러오기
- if __name__ == "__main__":의 의미
- 클래스나 변수 등을 포함한 모듈
- 다른 파일에서 모듈 불러오기
- 다른 디렉터리에 있는 모듈을 불러오는 방법
    - sys.path.append 사용하기
    - PYTHONPATH 환경 변수 사용하기

모듈 만들기

모듈에 대해 자세히 살펴보기 전에 간단한 모듈을 한번 만들어 보자.

```
# mod1.py
def add(a, b):
    return a + b

def sub(a, b):
    return a-b
```

위와 같이 add와 sub 함수만 있는 파일 mod1.py를 만들고 `C:\doit` 디렉터리에 저장하자. 이 mod1.py 파일이 바로 모듈이다. 지금까지 에디터로 만든 파이썬 파일과 다르지 않다.

> *파이썬 확장자 .py로 만든 파이썬 파일은 모두 모듈이다.*

모듈 불러오기

mod1.py 파일, 즉 모듈을 파이썬에서 불러와 사용하려면 어떻게 해야 할까?

먼저 명령 프롬프트 창을 열고 mod1.py를 저장한 디렉터리(이 책에서는 `C:\doit`)로 이동한 후 대화형 인터프리터를 실행해 보자.

```
C:\Users\pahkey>cd C:\doit
C:\doit>dir
 ...
2014-09-23 오후  01:53 49 mod1.py
 ...
C:\doit>python
>>>
```

반드시 mod1.py 파일을 저장한 `C:\doit` 디렉터리로 이동한 후 예제를 진행해야 한다. 그래야만 대화형 인터프리터에서 mod1.py 모듈을 읽을 수 있다.

다음과 같이 따라 해 보자.

```
>>> import mod1
>>> print(mod1.add(3, 4))
7
>>> print(mod1.sub(4, 2))
2
```

mod1.py 모듈을 불러오기 위해 `import mod1` 이라고 입력했다. 실수로 `import mod1.py` 라고 입력하지 않도록 주의하자. import는 이미 만들어 놓은 파이썬 모듈을 사용할 수 있게 해 준다. mod1.py 파일에 있는 add 함수를 사용하려면 mod1.add 처럼 모듈 이름 뒤에 도트 연산자(.)를 붙이고 함수 이름을 쓰면 된다.

> - *import는 현재 디렉터리에 있는 파일이나 파이썬 라이브러리가 저장된 디렉터리에 있는 모듈만 불러올 수 있다.*
> - *파이썬 라이브러리는 파이썬을 설치할 때 자동으로 설치되는 파이썬 모듈을 말한다.*

import의 사용 방법은 다음과 같다.

```
import 모듈_이름
```

여기에서 모듈 이름은 mod1.py에서 .py 확장자를 제거한 mod1만을 가리킨다.

때로는 mod1.add, mod1.sub처럼 쓰지 않고 add, sub처럼 모듈 이름 없이 함수 이름만 쓰고 싶을 수 있다. 이럴 때는 다음과 같이 사용하면 된다.

```
from 모듈_이름 import 모듈_함수
```

위와 같이 함수를 직접 import하면 모듈 이름을 붙이지 않고 바로 해당 모듈의 함수를 쓸 수 있다.

다음과 같이 따라 해 보자.

```
>>> from mod1 import add
>>> add(3, 4)
7
```

그런데 이렇게 하면 mod1.py 파일의 add 함수만 사용할 수 있다. add 함수와 sub 함수를 모두 모듈 이름 없이 사용하려면 어떻게 해야 할까?

2가지 방법이 있다.

```
from mod1 import add, sub
```

첫 번째 방법은 위와 같이 `from 모듈_이름 import 모듈_함수1, 모듈_함수2` 처럼 사용하는 것이다. 쉼표(,)로 구분하여 필요한 함수를 불러올 수 있다.

```
from mod1 import *
```

두 번째 방법은 `*` 문자를 사용하는 것이다. `*` 문자는 '모든 것'이라는 의미로, 파이썬에서도 같은 의미로 사용한다. 따라서 `from mod1 import *`은 mod1 모듈의 모든 함수를 불러와 사용하겠다는 뜻이다.

mod1.py 파일에는 함수가 2개밖에 없으므로 위 2가지 방법은 동일한 결과를 가져온다.

`if __name__ == "__main__":`의 의미

이번에는 mod1.py 파일을 다음과 같이 수정해 보자.

```
# mod1.py
def add(a, b):
    return a+b

def sub(a, b):
    return a-b

[[MARK]]print(add(1, 4))
print(sub(4, 2))[[/MARK]]
```

`add(1, 4)`와 `sub(4, 2)`의 결과를 출력하는 문장을 추가했다. 출력한 결괏값을 확인하기 위해 mod1.py 파일을 실행해 보자.

```
C:\doit>python mod1.py
5
2
```

예상한 대로 결괏값이 잘 출력된다. 그런데 이 mod1.py 파일의 add와 sub 함수를 사용하기 위해 mod1 모듈을 import할 때는 이상한 문제가 생긴다. 명령 프롬프트 창에서 다음을 따라 해 보자.

```
C:\Users\pahkey> cd C:\doit
C:\doit> python
>>> import mod1
[[MARK]]5
2[[/MARK]]
```

import mod1을 수행하는 순간 mod1.py 파일이 실행되어 결괏값을 출력한다. 단지 mod1.py 파일의 add와 sub 함수만 사용하려고 했는데 말이다.

이 문제를 방지하려면 mod1.py 파일을 다음처럼 수정해야 한다.

```
# mod1.py
def add(a, b):
    return a+b

def sub(a, b):
    return a-b

[[MARK]]if __name__ == "__main__":[[/MARK]]
    print(add(1, 4))
    print(sub(4, 2))
```

`if __name__ == "__main__"`을 사용하면 `C:\doit>python mod1.py`처럼 직접 이 파일을 실행했을 때는 `__name__ == "__main__"`이 참이 되어 if 문 다음 문장이 수행된다. 반대로 대화형 인터프리터나 다른 파일에서 이 모듈을 불러 사용할 때는 `__name__ == "__main__"`이 거짓이 되어 if 문 다음 문장이 수행되지 않는다.

위와 같이 수정한 후 다시 대화형 인터프리터를 열고 실행해 보자.

```
>>> import mod1
>>>
```

아무런 결괏값도 출력되지 않는 것을 확인할 수 있다.

점프 투 파이썬

`__name__` 변수란?

파이썬의 `__name__` 변수는 파이썬이 내부적으로 사용하는 특별한 변수 이름이다. `C:\doit>python mod1.py`처럼 직접 mod1.py 파일을 실행할 경우, mod1.py의 `__name__` 변수에는 `__main__` 값이 저장된다. 하지만 파이썬 셸이나 다른 파이썬 모듈에서 mod1을 import할 경우에는 mod1.py의 `__name__` 변수에 mod1.py의 모듈 이름인 mod1이 저장된다.

```
>>> import mod1
>>> mod1.__name__
'mod1'
```

클래스나 변수 등을 포함한 모듈

지금까지 살펴본 모듈은 함수만 포함했지만, 클래스나 변수 등을 포함할 수도 있다. 다음 프로그램을 작성해 보자.

```
# mod2.py
PI = 3.141592

class Math:
    def solv(self, r):
        return PI * (r ** 2)

def add(a, b):
    return a+b
```

이 파일은 원의 넓이를 계산하는 Math 클래스, 두 값을 더하는 add 함수, 원주율 값에 해당하는 PI 변수를 포함하고 있다.

파일 이름을 mod2.py로 하고 `C:\doit` 디렉터리에 저장하자. 그리고 대화형 인터프리터를 열어 다음과 같이 따라 해 보자.

```
C:\Users\pahkey> cd C:\doit
C:\doit> python
>>> import mod2
>>> print(mod2.PI)
3.141592
```

위 예에서 볼 수 있듯이 `mod2.PI`를 입력해서 mod2.py 파일에 있는 PI 변수의 값을 사용할 수 있다.

```
>>> a = mod2.Math()
>>> print(a.solv(2))
12.566368
```

위 예는 mod2.py에 있는 Math 클래스를 사용하는 방법을 보여 준다.

```
>>> print(mod2.add(mod2.PI, 4.4))
7.541592
```

mod2.py에 있는 add 함수 역시 당연히 사용할 수 있다.

다른 파일에서 모듈 불러오기

지금까지는 모듈 파일을 사용하기 위해 대화형 인터프리터만 사용했다. 이번에는 다른 파이썬 파일에서 모듈을 불러와서 사용하는 방법을 알아보자. 여기에서는 조금 전에 만든 mod2.py 파일을 불러올 것이다.

먼저 에디터로 `C:\doit\modtest.py` 파일을 생성하고 다음과 같이 작성하자.

```
# modtest.py
import mod2
result = mod2.add(3, 4)
print(result)
```

위에서 볼 수 있듯이 다른 파이썬 파일에서도 `import mod2` 로 mod2 모듈을 불러와서 사용할 수 있다. 대화형 인터프리터에서 한 것과 동일하다. 위 예제가 정상적으로 실행되려면 modtest.py 파일과 mod2.py 파일이 동일한 디렉터리(`C:\doit`)에 있어야 한다.

다른 디렉터리에 있는 모듈을 불러오는 방법

지금까지는 해당 모듈이 있는 디렉터리로 이동한 후에야 그 모듈을 사용할 수 있었다. 이번에는 모듈을 저장한 디렉터리로 이동하지 않고 모듈을 불러와서 사용하는 방법을 알아보자.

먼저 이전에 만든 mod2.py 파일을 `C:\doit\mymod` 로 이동시킨다.

```
C:\Users\pahkey>cd C:\doit
C:\doit>mkdir mymod
C:\doit>move mod2.py mymod
        1개 파일을 이동했습니다.
```

> *`mkdir mymod` 명령어는 mymod라는 디렉터리를 생성하라는 터미널 명령어이다.*

그리고 다음 예를 따라 해 보자.

sys.path.append 사용하기

먼저 파이썬 셸을 실행한 후 sys 모듈을 불러온다.

```
C:\doit>python
>>> import sys
```

sys 모듈은 파이썬을 설치할 때 함께 설치되는 라이브러리 모듈이다. 이 sys 모듈을 사용하면 파이썬 라이브러리가 설치되어 있는 디렉터리를 확인할 수 있다.

다음과 같이 입력해 보자.

```
>>> sys.path
['', 'C:\\Windows\\SYSTEM32\\python311.zip', 'c:\\Python311\\DLLs',
'c:\\Python311\\lib', 'c:\\Python311', 'c:\\Python311\\lib\\site-packages']
```

sys.path는 파이썬 라이브러리가 설치되어 있는 디렉터리 목록을 보여 준다. 이 디렉터리 안에 저장된 파이썬 모듈은 모듈이 저장된 디렉터리로 이동할 필요 없이 바로 불러 사용할 수 있다.

그렇다면 sys.path에 `C:\doit\mymod` 디렉터리를 추가하면 mymod 디렉터리에 저장된 파이썬 모듈을 어디서든 불러 사용할 수 있지 않을까? 맞다. sys.path는 리스트이므로 다음과 같이 할 수 있다.

```
>>> sys.path.append("C:/doit/mymod")
>>> sys.path
['', 'C:\\Windows\\SYSTEM32\\python311.zip', 'c:\\Python311\\DLLs',
'c:\\Python311\\lib', 'c:\\Python311', 'c:\\Python311\\lib\\site-packages',
[[MARK]]'C:/doit/mymod'[[/MARK]]]
>>>
```

sys.path.append를 사용해서 `C:/doit/mymod` 디렉터리를 sys.path에 추가했다. 다시 sys.path를 출력해 보니 가장 마지막에 `C:/doit/mymod` 디렉터리가 추가된 것을 확인할 수 있다.

실제로 디렉터리 이동 없이 바로 모듈을 불러와서 사용할 수 있는지 확인해 보자.

```
>>> import mod2
>>> print(mod2.add(3,4))
7
```

이상 없이 불러와서 사용할 수 있다.

PYTHONPATH 환경 변수 사용하기

모듈을 불러오는 또 다른 방법으로 `PYTHONPATH` 환경 변수를 사용할 수 있다.

다음과 같이 따라 해 보자.

```
C:\doit>set PYTHONPATH=C:\doit\mymod
C:\doit>python
>>> import mod2
>>> print(mod2.add(3, 4))
7
```

맥/리눅스에서는 다음처럼 실행한다.

```
$ export PYTHONPATH=~/doit/mymod
$ python3
>>> import mod2
>>> print(mod2.add(3, 4))
7
```

set 명령어를 사용해 `PYTHONPATH` 환경 변수에 mod2.py 파일이 있는 `C:\doit\mymod` 디렉터리를 설정한다. 그러면 디렉터리 이동이나 별도의 모듈 추가 작업 없이 mymod 디렉터리에 저장된 mod2 모듈을 불러와서 사용할 수 있다.

> *맥이나 유닉스 환경에서는 set 대신 export 명령을 사용해야 한다.*

05-3 패키지

패키지(packages)란 관련 있는 모듈의 집합이다. 패키지를 사용하면 파이썬 모듈을 계층적(디렉터리 구조)으로 관리할 수 있다.

> *파이썬에서 모듈은 하나의 .py 파일이다.*

컴퓨터에서 파일을 관리할 때 관련 있는 파일끼리 폴더로 묶어 정리하면 찾기 쉽고 관리하기 편하다. 파이썬의 패키지도 이와 같은 개념이다. 모듈이 많아지면 이름이 겹칠 수도 있고 관리가 어려워지는데, 패키지를 사용하면 관련 모듈을 폴더별로 깔끔하게 정리할 수 있다.

파이썬 패키지는 디렉터리와 파이썬 모듈로 이루어진다. 다음은 game이라는 가상의 파이썬 패키지 구조이다.

가상의 game 패키지 예

```
game/
    __init__.py
    sound/
        __init__.py
        echo.py
        wav.py
    graphic/
        __init__.py
        screen.py
        render.py
    play/
        __init__.py
        run.py
        test.py
```

game, sound, graphic, play는 디렉터리이고, 확장자가 .py인 파일은 파이썬 모듈이다. game 디렉터리가 이 패키지의 루트 디렉터리이고, sound, graphic, play는 서브 디렉터리이다.

간단한 프로그램이 아니라면 패키지 구조로 만드는 것이 공동 작업이나 유지 보수에 유리하다. 또한 패키지 구조로 모듈을 만들면 다른 모듈과 이름이 겹치더라도 안전하게 사용할 수 있다.

- 패키지 만들기
- 패키지 안의 함수 실행하기
- __init__.py 파일의 용도
  - 패키지 변수 및 함수 정의
  - 패키지 내 모듈을 미리 import
  - 패키지 초기화
  - __all__
- 상대경로 패키지

패키지 만들기

이제 위 예와 비슷한 game 패키지를 직접 만들어 보자.

1. `C:/doit` 디렉터리 밑에 game 및 서브 디렉터리를 생성하고 .py 파일들을 다음과 같이 만든다(만약 `C:/doit` 디렉터리가 없다면 먼저 생성하고 진행하자).

```
C:/doit/game/__init__.py
C:/doit/game/sound/__init__.py
C:/doit/game/sound/echo.py
C:/doit/game/graphic/__init__.py
C:/doit/game/graphic/render.py
```

2. 각 디렉터리에 `__init__.py` 파일을 만들고 내용은 비워 둔다. `__init__.py` 파일이 있어야 파이썬이 해당 디렉터리를 패키지로 인식한다. 지금은 내용을 비워 두지만, 뒤에서 이 파일의 다양한 활용법을 알아본다.

3. echo.py 파일은 다음과 같이 작성한다.

```
# echo.py
def echo_test():
    print("echo")
```

4. render.py 파일은 다음과 같이 작성한다.

```
# render.py
def render_test():
    print("render")
```

5. 다음 예제를 수행하기 전에 game 패키지를 참조할 수 있도록 명령 프롬프트 창에서 set 명령어로 PYTHONPATH 환경 변수에 `C:/doit` 디렉터리를 추가하고 파이썬 인터프리터를 실행한다.

> *PYTHONPATH는 파이썬이 모듈을 찾을 때 추가로 살펴볼 디렉터리를 알려주는 환경 변수이다. 여기서는 `C:/doit` 디렉터리를 PYTHONPATH에 추가하여 그 안에 있는 game 패키지를 어디서든 import할 수 있게 한다.*

```
C:\> set PYTHONPATH=C:/doit
C:\> python
>>>
```

맥/리눅스에서는 다음처럼 실행한다.

```
$ export PYTHONPATH=~/doit
$ python3
>>>
```

여기까지 준비가 되었다면 다음을 따라 해 보자.

점프 투 파이썬

중요 공지

다음에 나올 실습은 반드시 명령 프롬프트에서 파이썬 인터프리터를 실행하여 진행해야 한다. 많은 독자가 IDLE 셸 또는 비주얼 스튜디오의 파이썬 셸에서 다음 예제를 실행하다가 오류를 만난다.

패키지 안의 함수 실행하기

이제 패키지를 사용하여 echo.py 파일의 echo_test 함수를 실행해 보자. 패키지 안의 함수를 실행하는 방법은 3가지가 있다.

점프 투 파이썬

인터프리터 재시작에 대하여

다음 import 예제들은 하나의 예제를 실행한 후 다음 예제를 실행할 때 반드시 인터프리터를 종료(`exit()` 입력)하고 다시 실행해야 한다. 파이썬은 한번 import한 모듈을 메모리에 저장해 두기 때문에, 인터프리터를 재시작하지 않으면 이전에 import한 것이 남아 있어 예상과 다른 결과가 나올 수 있다. 이 규칙은 이 장의 모든 예제에 동일하게 적용된다.

첫 번째는 echo 모듈을 import하여 실행하는 방법이다.

> *echo 모듈은 echo.py 파일이다.*

```
>>> import game.sound.echo
>>> game.sound.echo.echo_test()
echo
```

두 번째는 echo 모듈이 있는 디렉터리까지를 `from ... import` 하여 실행하는 방법이다. 인터프리터를 재시작한 후 다음 코드를 입력하자.

```
>>> exit()
C:\> python
>>> from game.sound import echo
>>> echo.echo_test()
echo
```

세 번째는 echo 모듈의 echo_test 함수를 직접 import하여 실행하는 방법이다.

```
>>> from game.sound.echo import echo_test
>>> echo_test()
echo
```

하지만 다음과 같은 방법은 불가능하다.

> *인터프리터를 재시작하고 진행하자.*

```
>>> import game
>>> game.sound.echo.echo_test()
Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
AttributeError: 'module' object has no attribute 'sound'
```

import game을 수행하면 game 디렉터리의 `__init__.py`에 정의된 것만 참조할 수 있다.

또한 다음과 같은 방법도 불가능하다.

```
>>> import game.sound.echo.echo_test
Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'game.sound.echo.echo_test'; 'game.sound.echo' is not a p
ackage
```

도트 연산자(.)를 사용해서 `import a.b.c`처럼 import할 때 마지막 항목인 c는 반드시 모듈 또는 패키지여야 한다.

`__init__.py` 파일의 용도

`__init__.py` 파일은 해당 디렉터리가 패키지의 일부임을 알려주는 역할을 한다. 만약 game, sound, graphic 등 패키지에 포함된 디렉터리에 `__init__.py` 파일이 없다면 패키지로 인식되지 않는다.

> *python 3.3 버전부터는 `__init__.py` 파일이 없어도 패키지로 인식한다. 하지만 `__init__.py` 파일을 생성하는 것이 파이썬 커뮤니티의 일반적인 관례이므로 항상 만들어 주는 것이 좋다.*

`__init__.py` 파일은 패키지 설정이나 초기화 코드를 포함할 수도 있다. 몇 가지 예를 살펴보자.

> *`__init__.py` 파일을 수정한 후에는 인터프리터를 재시작해야 한다.*

패키지 변수 및 함수 정의

패키지 수준에서 변수와 함수를 정의할 수 있다. 예를 들어 game 패키지의 `__init__.py` 파일에 공통 변수나 함수를 정의할 수 있다.

```
# C:/doit/game/__init__.py
VERSION = 3.5

def print_version_info():
    print(f"The version of this game is {VERSION}.")
```

이렇게 정의된 변수와 함수는 다음과 같이 사용할 수 있다.

```
>>> import game
>>> print(game.VERSION)
3.5
>>> game.print_version_info()
The version of this game is 3.5.
```

패키지 내 모듈을 미리 import

`__init__.py` 파일에 패키지 내의 다른 모듈을 미리 import하면 패키지 사용 시 간편하게 접근할 수 있다.

```
# C:/doit/game/__init__.py
[[MARK]]from .graphic.render import render_test[[/MARK]]

VERSION = 3.5

def print_version_info():
    print(f"The version of this game is {VERSION}.")
```

> *`from .graphic.render import render_test` 문장에서 `.graphic.render`의 맨 앞 `.`은 현재 디렉터리를 의미한다. 이에 대해서는 뒤에서 자세히 알아본다.*

이제 다음과 같이 game 패키지를 통해 render_test 함수를 간편하게 사용할 수 있다.

```
>>> import game
>>> game.render_test()
render
```

패키지 초기화

`__init__.py` 파일에 패키지를 처음 불러올 때 실행할 코드를 작성할 수 있다. 예를 들어 데이터베이스 연결이나 설정 파일 로드 같은 작업을 수행할 수 있다.

```
# C:/doit/game/__init__.py
from .graphic.render import render_test

VERSION = 3.5

def print_version_info():
    print(f"The version of this game is {VERSION}.")

[[MARK]]# 여기에 패키지 초기화 코드를 작성한다.
print("Initializing game ...")[[/MARK]]
```

이렇게 하면 패키지를 처음 import할 때 초기화 코드가 실행된다.

```
>>> import game
Initializing game ...
>>>
```

game 패키지의 초기화 코드는 하위 모듈의 함수를 import할 경우에도 실행된다.

```
>>> from game.graphic.render import render_test
Initializing game ...
>>>
```

단, 초기화 코드는 한 번 실행된 후에는 다시 import해도 실행되지 않는다. 예를 들어 game 패키지를 import한 후 하위 모듈을 다시 import해도 초기화 코드는 처음 한 번만 실행된다.

```
>>> import game
Initializing game ...
>>> from game.graphic.render import render_test
>>>
```

`__all__`

다음을 따라 해 보자.

```
>>> from game.sound import *
Initializing game  ...
>>> echo.echo_test()
Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
NameError: name 'echo' is not defined
```

예상과 다른 결과다. game.sound 패키지에서 모든 것(`*`)을 import했으므로 echo 모듈을 사용할 수 있어야 할 것 같은데, echo라는 이름이 정의되지 않았다는 오류가 발생했다.

이런 오류가 발생하는 이유는, 디렉터리(패키지)에서 `*` 를 사용하여 import하면 파이썬은 해당 디렉터리 안에 어떤 모듈을 가져와야 할지 스스로 판단하지 못하기 때문이다. 그래서 `__all__` 변수를 사용하여 "이 디렉터리에서 `*` 로 import할 때 가져올 모듈 목록"을 직접 알려줘야 한다.

해당 디렉터리의 `__init__.py` 파일에 `__all__` 변수를 다음과 같이 설정하자.

```
# C:/doit/game/sound/__init__.py
__all__ = ['echo']
```

이렇게 하면 `from game.sound import *` 를 수행할 때 `__all__` 에 정의된 echo 모듈을 import하게 된다.

> *착각하기 쉬운데 `from game.sound.echo import *` 는 `__all__` 과 상관없이 import된다. 이 경우 from의 마지막 항목인 echo가 모듈(파일)이므로, 파이썬이 그 파일 안에 있는 모든 함수를 직접 가져올 수 있기 때문이다. `__all__` 이 필요한 경우는 `from game.sound import *` 처럼 from의 마지막 항목이 디렉터리(패키지)인 때뿐이다.*

위와 같이 `__init__.py` 파일을 변경한 후 예제를 수행하면 원하는 결과가 출력된다.

```
>>> from game.sound import *
Initializing game  ...
>>> echo.echo_test()
echo
```

상대경로 패키지

graphic 디렉터리의 render.py 모듈에서 sound 디렉터리의 echo.py 모듈을 사용하고 싶다면 어떻게 해야 할까? 다음과 같이 render.py를 수정하면 된다.

```
# render.py
from game.sound.echo import echo_test
def render_test():
    print("render")
    echo_test()
```

`from game.sound.echo import echo_test` 문장을 추가하여 echo_test 함수를 사용할 수 있도록 수정했다.

수정한 후 다음과 같이 실행해 보자.

```
>>> from game.graphic.render import render_test
Initializing game  ...
>>> render_test()
render
echo
```

잘 수행된다.

위 예제처럼 `from game.sound.echo import echo_test`로 전체 경로를 사용하여 import할 수도 있지만, 다음과 같이 상대경로로 import할 수도 있다.

```
# render.py
from ..sound.echo import echo_test

def render_test():
    print("render")
    echo_test()
```

`from game.sound.echo import echo_test`를 `from ..sound.echo import echo_test`로 수정했다. 여기서 `..`은 render.py 파일의 부모 디렉터리를 의미한다. render.py 파일의 부모 디렉터리는 game이므로 이와 같은 import가 가능하다.

render.py 기준으로 `.`과 `..`이 어디를 가리키는지 다음 그림을 보면 쉽게 이해할 수 있다.

```
game/                ← .. (부모 디렉터리)
    __init__.py
    sound/
        __init__.py
        echo.py      ← ..sound.echo로 접근
    graphic/         ← .  (현재 디렉터리)
        __init__.py
        render.py    ← 여기서 import
```

상대경로 접근자는 다음과 같다.

접근자	설명
`..`	부모 디렉터리
`.`	현재 디렉터리

05-4 예외 처리

프로그램을 만들다 보면 수없이 많은 오류를 만나게 된다. 물론 오류가 발생하는 이유는 프로그램이 잘못 동작하는 것을 막기 위한 파이썬의 배려이다. 이번에는 파이썬에서 오류를 처리하는 방법에 대해 알아보자.

- 오류는 언제 발생하는가?
- 오류 예외 처리 기법
    - try-except 문
    - try-finally 문
    - 여러 개의 오류 처리하기
    - try-else 문
- 오류 회피하기
- 오류 일부러 발생시키기
- 예외 만들기

오류는 언제 발생하는가?

오류를 처리하는 방법을 공부하기 전에 어떤 상황에서 오류가 발생하는지 알아보자. 오타를 입력했을 때 발생하는 구문 오류가 아닌, 실제 프로그램에서 자주 발생하는 오류를 중심으로 살펴보자.

먼저 존재하지 않는 파일을 열려고 할 때 발생하는 오류이다.

```
>>> f = open("나없는파일", 'r')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: '나없는파일'
```

위 예에서 볼 수 있듯이 없는 파일을 열려고 시도하면 FileNotFoundError 오류가 발생한다.

이번에는 0으로 다른 숫자를 나누는 경우를 살펴보자.

```
>>> 4 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

4를 0으로 나누려고 하니 ZeroDivisionError 오류가 발생한다.

마지막으로 한 가지 예를 더 들어 보자. 다음 오류는 정말 자주 발생한다.

```
>>> a = [1, 2, 3]
>>> a[3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

a[3]은 a의 네 번째 요솟값을 가리키는데, a 리스트에는 값이 3개밖에 없으므로([1, 2, 3]) 값을 얻을 수 없다. 따라서 IndexError 오류가 발생한다. 파이썬은 이런 오류가 발생하면 프로그램을 중단하고 오류 메시지를 보여 준다.

오류 예외 처리 기법

이제 유연한 프로그래밍을 위한 오류 처리 방법에 대해 알아보자.

try-except 문

다음은 오류를 처리하기 위한 try-except 문의 기본 구조이다.

```
try:
    ...
except [발생오류 [as 오류변수]]:
    ...
```

try 블록 수행 중 오류가 발생하면 except 블록이 수행된다. 하지만 try 블록에서 오류가 발생하지 않는다면 except 블록은 수행되지 않는다.

except 구문을 자세히 살펴보자.

```
except [발생오류 [as 오류변수]]:
```

위 구문에서 `[]`는 괄호 안의 내용을 생략할 수 있다는 관례적인 표기법이다. 파이썬의 리스트를 뜻하는 대괄호가 아니라, 문법을 설명할 때 "이 부분은 써도 되고 안 써도 된다"는 의미이다. 즉, except 구문은 다음 3가지 방법으로 사용할 수 있다.

1. try-except만 쓰는 방법

```
try:
    ...
except:
    ...
```

이 경우에는 오류의 종류에 상관없이 오류가 발생하면 except 블록을 수행한다.

2. 발생 오류만 포함한 except 문

```
try:
    ...
except 발생오류:
    ...
```

이 경우는 오류가 발생했을 때 except 문에 지정한 오류와 동일한 오류일 경우에만 except 블록을 수행한다.

3. 발생 오류와 오류 변수까지 포함한 except 문

```
try:
    ...
except 발생오류 as 오류변수:
    ...
```

이 경우는 두 번째 경우에서 오류의 내용까지 알고 싶을 때 사용한다.

다음은 이 방법을 사용한 예이다.

```
# try_except.py
try:
    4 / 0
except ZeroDivisionError as e:
    print(e)
```

위처럼 4를 0으로 나누려고 하면 ZeroDivisionError가 발생하여 except 블록이 실행되고, 오류 변수 e에 담긴 오류 메시지를 출력할 수 있다. 출력되는 오류 메시지는 다음과 같다.

```
division by zero
```

try-finally 문

try 문에는 finally 절을 사용할 수 있다. finally 절은 try 문 수행 도중 예외 발생 여부에 상관없이 항상 수행된다. 보통 finally 절은 사용한 리소스를 close해야 할 때 많이 사용한다.

다음 예를 살펴보자.

```
# try_finally.py
try:
    f = open('foo.txt', 'w')
    # 무언가를 수행한다.

    (... 생략 ...)

finally:
    f.close()  # 중간에 오류가 발생하더라도 무조건 실행된다.
```

foo.txt 파일을 쓰기 모드로 연 후 예외 발생 여부에 상관없이 항상 파일을 닫아 주려면 try-finally 문을 사용하면 된다.

finally 절이 실제로 어떻게 동작하는지 좀 더 구체적인 예를 살펴보자.

```
# try_finally2.py
try:
    print("나누기 전")
    4 / 0
    print("나누기 후")
except ZeroDivisionError:
    print("오류가 발생했습니다.")
finally:
    print("finally 실행!")
```

위 코드를 실행하면 다음과 같이 출력된다.

```
나누기 전
오류가 발생했습니다.
finally 실행!
```

`4 / 0`에서 오류가 발생했지만 finally 절은 실행된 것을 확인할 수 있다. 오류가 발생하지 않더라도 finally 절은 항상 실행된다.

여러 개의 오류 처리하기

try 문 안에서 여러 개의 오류를 처리하려면 다음과 같이 작성한다.

```
try:
    ...
except 발생오류1:
   ...
except 발생오류2:
   ...
```

즉, 0으로 나누는 오류와 인덱싱 오류를 다음과 같이 처리할 수 있다.

```
# many_error.py
try:
    a = [1,2]
    print(a[3])
    4/0
except ZeroDivisionError:
    print("0으로 나눌 수 없습니다.")
except IndexError:
    print("인덱싱 할 수 없습니다.")
```

a는 2개의 요솟값을 가지고 있으므로 a[3]이 IndexError를 발생시켜 "인덱싱 할 수 없습니다."라는 문자열을 출력한다. 인덱싱 오류가 먼저 발생했으므로 `4 / 0`에 따른 ZeroDivisionError 오류는 발생하지 않는다.

앞에서 알아본 것과 마찬가지로 오류 메시지도 다음과 같이 확인할 수 있다.

```
try:
    a = [1,2]
    print(a[3])
    4/0
except ZeroDivisionError as e:
    print(e)
except IndexError as e:
    print(e)
```

프로그램을 실행하면 'list index out of range'라는 오류 메시지가 출력될 것이다.

다음과 같이 ZeroDivisionError와 IndexError를 함께 처리할 수도 있다.

```
try:
    a = [1,2]
    print(a[3])
    4/0
except (ZeroDivisionError, IndexError) as e:
    print(e)
```

2개 이상의 오류를 동일하게 처리하려면 위와 같이 괄호를 사용하여 함께 묶어 처리하면 된다.

try-else 문

try 문에는 다음처럼 else 절을 사용할 수도 있다.

```
try:
    ...
except [발생오류 [as 오류변수]]:
    ...
else:  # 오류가 없을 경우에만 수행
    ...
```

try 문 수행 중 오류가 발생하면 except 절이, 오류가 발생하지 않으면 else 절이 수행된다.

> *"try 블록 안에 그냥 넣으면 되지 않나?"라고 생각할 수 있다. 하지만 else 절에 넣은 코드는 except의 대상이 되지 않는다. 따라서 try 블록에서 발생할 수 있는 오류만 정확히 잡고, 나머지 코드에서 발생하는 의도치 않은 오류가 except에 잡히는 것을 방지할 수 있다.*

다음은 try 문에 else 절을 사용한 간단한 예제이다.

```
# try_else.py
try:
    age=int(input('나이를 입력하세요: '))
except:
    print('입력이 정확하지 않습니다.')
else:
    if age ⩽ 18:
        print('미성년자는 출입금지입니다.')
    else:
        print('환영합니다.')
```

'나이를 입력하세요: '라는 질문에 숫자가 아닌 다른 값을 입력하면 오류가 발생하여 '입력이 정확하지 않습니다.'라는 문장을 출력한다. 오류가 없을 경우에만 else 절이 수행된다.

오류 회피하기

코드를 작성하다 보면 특정 오류가 발생해도 그냥 통과시켜야 할 때가 있다. 예를 들어, 여러 파일을 처리하는 중에 일부 파일이 없더라도 프로그램을 계속 실행하고 싶을 때가 있다.

다음은 학생들의 성적 파일을 읽어서 처리하는 예제이다.

```
# process_scores.py
students = ["김철수", "이영희", "박민수", "최유진"]

for student in students:
    try:
        with open(f"{student}_성적.txt", 'r') as f:
            score = f.read()
            print(f"{student}의 성적: {score}")
    except FileNotFoundError:
        print(f"{student}의 성적 파일이 없습니다. 건너뜁니다.")
        continue  # 다음 학생으로 넘어감
```

위 코드에서 일부 학생의 성적 파일이 없어도 프로그램이 중단되지 않고 다른 학생들의 성적을 계속 처리할 수 있다.

때로는 오류를 완전히 무시하고 싶을 때도 있다. 이럴 때 pass를 사용한다.

```
# error_pass.py
try:
    # 설정 파일을 읽으려 시도
    with open("설정파일.txt", 'r') as f:
        config = f.read()
except FileNotFoundError:
    pass  # 설정 파일이 없어도 계속 진행

# 프로그램의 주요 기능은 계속 수행
print("프로그램이 정상적으로 실행됩니다.")
```

pass를 사용하면 오류가 발생해도 프로그램이 중단되지 않고 계속 진행된다. 하지만 pass는 신중하게 사용해야 한다. 중요한 오류까지 무시하면 나중에 더 큰 문제가 될 수 있기 때문이다.

오류 일부러 발생시키기

이상하게 들리겠지만, 프로그래밍을 하다 보면 오류를 일부러 발생시켜야 할 경우도 있다. 파이썬은 raise 문을 사용해 오류를 강제로 발생시킬 수 있다.

왜 일부러 오류를 발생시킬까? 예를 들어 여러 명이 함께 작업할 때, 자식 클래스가 반드시 구현해야 할 기능을 깜빡하고 빠뜨리는 실수를 방지하기 위해서이다. 오류를 일부러 발생시키면 "이 기능을 아직 구현하지 않았다"는 사실을 바로 알 수 있다.

예를 들어 Bird 클래스를 상속받는 자식 클래스는 반드시 fly라는 함수를 구현하도록 만들고 싶은 경우가 있을 수 있다. 다음 예를 살펴보자.

```
# error_raise.py
class Bird:
    def fly(self):
        raise NotImplementedError
```

Bird 클래스를 상속받는 자식 클래스는 반드시 fly 함수를 구현해야 한다는 의지를 보여 준다. 만약 자식 클래스가 fly 함수를 구현하지 않은 상태로 fly 함수를 호출한다면 어떻게 될까?

> *NotImplementedError는 파이썬에 이미 정의되어 있는 오류로, 꼭 작성해야 하는 부분이 구현되지 않았을 경우 일부러 오류를 발생시키기 위해 사용한다.*

```
class Eagle(Bird):
    pass

eagle = Eagle()
eagle.fly()
```

Eagle 클래스는 Bird 클래스를 상속받았다. 그런데 Eagle 클래스는 fly 메서드를 오버라이딩하여 구현하지 않았다. 따라서 eagle 객체의 fly 메서드를 수행하는 순간 Bird 클래스의 fly 메서드가 수행되어 NotImplementedError가 발생한다.

```
Traceback (most recent call last):
  File "...", line 33, in <module>
    eagle.fly()
  File "...", line 26, in fly
    raise NotImplementedError
NotImplementedError
```

> *상속받는 클래스에서 메서드를 재구현하는 것을 '메서드 오버라이딩'이라고 한다.*

NotImplementedError가 발생하지 않게 하려면 다음과 같이 Eagle 클래스에 fly 함수를 구현해야 한다.

```
class Eagle(Bird):
    def fly(self):
        print("very fast")

eagle = Eagle()
eagle.fly()
```

위 예처럼 fly 함수를 구현한 후 프로그램을 실행하면 오류 없이 다음 문장이 출력된다.

```
very fast
```

예외 만들기

프로그램을 수행하다가 특수한 경우에만 예외 처리를 하려고 종종 예외를 직접 만들어서 사용한다. 이번에는 직접 예외를 만들어 보자.

예외는 다음과 같이 파이썬 내장 클래스인 Exception 클래스를 상속하여 만들 수 있다.

```
# error_make.py
class MyError(Exception):
    pass
```

그리고 별명을 출력하는 함수를 다음과 같이 작성해 보자.

```
def say_nick(nick):
    if nick == '바보':
        raise MyError()
    print(nick)
```

다음과 같이 say_nick 함수를 호출해 보자.

```
say_nick("천사")
say_nick("바보")
```

저장한 후 프로그램을 실행해 보면 다음과 같이 "천사"가 한 번 출력된 후 MyError가 발생한다.

```
천사
Traceback (most recent call last):
  File "...", line 11, in <module>
    say_nick("바보")
  File "...", line 7, in say_nick
    raise MyError()
__main__.MyError
```

이번에는 예외 처리 기법을 사용하여 MyError 발생을 예외 처리해 보자.

```
try:
    say_nick("천사")
    say_nick("바보")
except MyError:
    print("허용되지 않는 별명입니다.")
```

프로그램을 실행하면 다음과 같이 출력된다.

```
천사
허용되지 않는 별명입니다.
```

만약 오류 메시지를 사용하고 싶다면 다음처럼 예외 처리를 하면 된다.

```
try:
    say_nick("천사")
    say_nick("바보")
except MyError as e:
    print(e)
```

하지만 프로그램을 실행해 보면 `print(e)`로 오류 메시지가 출력되지 않는 것을 확인할 수 있다. MyError 클래스에 별도의 오류 메시지를 정의하지 않았으므로 빈 문자열이 출력되기 때문이다. 오류 메시지를 출력했을 때 오류 메시지가 보이게 하려면 오류 클래스에 다음과 같은 `__str__` 메서드를 구현해야 한다. `__str__` 메서드는 `print(e)`처럼 오류 메시지를 print 문으로 출력할 경우에 호출되는 메서드이다.

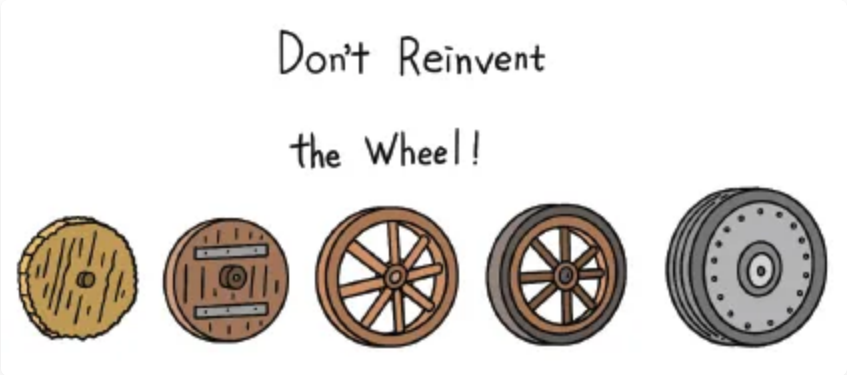
```
class MyError(Exception):
    def __str__(self):
        return "허용되지 않는 별명입니다."
```

프로그램을 다시 실행해 보면 "허용되지 않는 별명입니다."라는 오류 메시지가 출력되는 것을 확인할 수 있을 것이다.

05-5 내장 함수

지금까지 파이썬으로 프로그래밍하기 위해 알아야 하는 것들을 대부분 공부했다. 이제 여러분은 원하는 프로그램을 직접 만들 수 있을 것이다. 하지만 그 전에 먼저 여러분이 만들려는 프로그램이 이미 만들어져 있는지 살펴보기 바란다.

물론 공부를 위해서라면 누군가 만들어 놓은 프로그램을 또 만들 수도 있다. 하지만 그런 목적이 아니라면 이미 만들어진 것을 다시 만드는 것은 불필요한 행동이다. 게다가 이미 만들어진 프로그램은 수많은 테스트를 거쳐 충분히 검증되어 있다. 따라서 새로운 프로그램을 만들기 전에는 이미 만들어진 것들, 그중에서도 특히 파이썬 배포본에 함께 들어 있는 파이썬 라이브러리를 살펴보는 것이 매우 중요하다.



"이미 있는 것을 다시 만드느라 시간을 낭비하지 말라."

라이브러리를 살펴보기 전에 파이썬 내장(built-in) 함수를 먼저 살펴보자. 우리는 이미 몇 가지 내장 함수를 배웠다. print, type 등이 바로 그것이다. 파이썬 내장 함수는 모듈과 달리 import가 필요하지 않기 때문에 아무런 설정 없이 바로 사용할 수 있다.

이 책에서는 활용 빈도가 높고 중요한 함수를 중심으로 알파벳 순서대로 간략히 정리했다. 파이썬으로 프로그래밍을 하기 위해 이 함수들을 지금 당장 모두 알아야 하는 것은 아니므로 가벼운 마음으로 천천히 살펴보자.

- abs
- all
- any
- chr
- dir
- divmod
- enumerate
- eval
- filter
- hex

- id
- input
- int
- isinstance
- len
- list
- map
- max
- min
- oct
- open
- ord
- pow
- range
    - 인수가 하나일 경우
    - 인수가 2개일 경우
    - 인수가 3개일 경우
- round
- sorted
- str
- sum
- tuple
- type
- zip

abs

abs(x)는 숫자를 입력받아 그 숫자의 절댓값을 반환하는 함수이다.

```
>>> abs(3)
3
>>> abs(-3)
3
>>> abs(-1.2)
1.2
```

all

all(x)는 반복 가능한 데이터 x를 입력값으로 받아 x의 요소가 모두 참이면 True, 거짓이 하나라도 있으면 False를 반환한다.

> *반복 가능한 데이터란 for 문에서 사용할 수 있는 자료형을 의미한다. 리스트, 튜플, 문자열, 딕셔너리, 집합 등이 있다.*

다음 예를 살펴보자.

```
>>> all([1, 2, 3])
True
```

리스트 [1, 2, 3]은 모든 요소가 참이므로 True를 반환한다.

```
>>> all([1, 2, 3, 0])
False
```

리스트 [1, 2, 3, 0] 중에서 요소 0은 거짓이므로 False를 반환한다.

```
>>> all([])
True
```

all의 입력 인수가 빈 값인 경우에는 True를 반환한다.

> *자료형의 참과 거짓은 02-7을 참고하기 바란다.*

any

any(x)는 반복 가능한 데이터 x를 입력으로 받아 x의 요소 중 하나라도 참이 있으면 True를 반환하고, x가 모두 거짓일 때만 False를 반환한다. 즉, all(x)의 반대로 작동한다.

다음 예를 살펴보자.

```
>>> any([1, 2, 3, 0])
True
```

리스트 [1, 2, 3, 0] 중에서 1, 2, 3이 참이므로 True를 반환한다.

```
>>> any([0, ""])
False
```

리스트 [0, ""]의 요소 0과 ""은 모두 거짓이므로 False를 반환한다.

```
>>> any([])
False
```

any의 입력 인수가 빈 값인 경우에는 False를 반환한다.

chr

chr(i)는 유니코드 숫자 값을 입력받아 그 코드에 해당하는 문자를 반환하는 함수이다.

> *유니코드는 전 세계의 모든 문자를 컴퓨터에서 일관되게 표현하고 다룰 수 있도록 설계된 산업 표준 코드이다.*

```
>>> chr(97)
'a'
>>> chr(44032)
'가'
```

dir

dir은 객체가 지닌 변수나 함수를 보여 주는 함수이다. 다음 예는 리스트와 딕셔너리가 지닌 함수(메서드)를 보여 준다. 02장에서 살펴본 함수들을 확인할 수 있다.

```
>>> dir([1, 2, 3])
['append', 'count', 'extend', 'index', 'insert', 'pop', ... ]
>>> dir({'1':'a'})
['clear', 'copy', 'get', 'items', 'keys', 'values', ... ]
```

divmod

divmod(a, b)는 2개의 숫자 a, b를 입력으로 받아 a를 b로 나눈 몫과 나머지를 튜플로 반환한다.

```
>>> divmod(7, 3)
(2, 1)
```

몫을 구하는 연산자 `//`와 나머지를 구하는 연산자 `%`를 각각 사용한 결과와 비교해 보자.

```
>>> 7 // 3
2
>>> 7 % 3
1
```

enumerate

enumerate는 '열거하다'라는 뜻이다. 이 함수는 순서가 있는 데이터(리스트, 튜플, 문자열)를 입력으로 받아 인덱스 값을 포함하는 enumerate 객체를 반환한다.

> *보통 enumerate 함수는 for 문과 함께 사용한다.*

다음 예를 살펴보자.

```
>>> for i, name in enumerate(['body', 'foo', 'bar']):
...     print(i, name)
...
0 body
1 foo
2 bar
```

인덱스 값과 함께 body, foo, bar가 순서대로 출력되었다. 즉, enumerate를 for 문과 함께 사용하면 자료형의 현재 순서(index)와 그 값을 쉽게 알 수 있다.

for 문처럼 반복되는 구간에서 객체가 현재 어느 위치에 있는지 알려 주는 인덱스 값이 필요할 때 enumerate 함수를 사용하면 매우 유용하다.

eval

eval(expression)은 문자열로 구성된 표현식을 입력으로 받아 해당 문자열을 실행한 결괏값을 반환하는 함수이다.

```
>>> eval('1+2')
3
>>> eval("'hi' + 'a'")
'hia'
>>> eval('divmod(4, 3)')
(1, 1)
```

> *eval은 입력 문자열을 실제로 실행하므로, 신뢰할 수 없는 외부 입력에는 사용하면 안 된다.*

filter

filter란 '무엇인가를 걸러 낸다'라는 뜻으로, filter 함수도 이와 비슷한 기능을 한다.

```
filter(함수, 반복_가능한_데이터)
```

filter 함수는 첫 번째 인수로 함수, 두 번째 인수로 그 함수에 차례로 들어갈 반복 가능한 데이터를 받는다. 반복 가능한 데이터의 요소를 순서대로 함수에 전달하여 반환값이 참인 것만 묶어서 반환한다.

다음 예를 살펴보자.

```
# positive.py
def positive(l):
    result = []
    for i in l:
        if i > 0:
            result.append(i)
    return result

print(positive([1,-3,2,0,-5,6]))
```

`[실행 결과]`

```
[1, 2, 6]
```

위에서 만든 positive는 리스트를 입력받아 각 요소를 판별해서 양수 값만 반환하는 함수이다.

filter 함수를 사용하면 위 내용을 다음과 같이 간단하게 작성할 수 있다.

```
# filter1.py
def positive(x):
    return x > 0

print(list(filter(positive, [1, -3, 2, 0, -5, 6])))
```

`[실행 결과]`

```
[1, 2, 6]
```

`filter(positive, [1, -3, 2, 0, -5, 6])`은 [1, -3, 2, 0, -5, 6]의 각 요솟값을 순서대로 positive 함수에 적용하여 반환값이 참인 것만 묶어서 반환한다. 1, 2, 6만 `x > 0` 조건에 참이므로 [1, 2, 6]이라는 결괏값이 출력된다.

> *list 함수는 filter 함수의 반환값을 리스트로 출력하기 위해 사용했다.*

이 예제는 lambda를 사용하면 더욱 간단해진다.

```
>>> list(filter(lambda x: x > 0, [1, -3, 2, 0, -5, 6]))
[1, 2, 6]
```

hex

hex(x)는 정수를 입력받아 16진수(hexadecimal) 문자열로 변환하여 반환하는 함수이다.

```
>>> hex(234)
'0×ea'
>>> hex(3)
'0×3'
```

id

id(object)는 객체를 입력받아 객체의 고유 주솟값(레퍼런스)을 반환하는 함수이다.

```
>>> a = 3
>>> id(3)
135072304
>>> id(a)
135072304
>>> b = a
>>> id(b)
135072304
```

위 예에서 3, a, b는 고유 주솟값이 모두 135072304이다. 즉, 3, a, b가 모두 같은 객체를 가리키고 있다.

id(4)를 입력하면 4는 3, a, b와 다른 객체이므로 당연히 다른 고유 주솟값이 출력된다.

```
>>> id(4)
135072292
```

> *id 값은 실행 환경과 시점에 따라 달라질 수 있다. 위 숫자는 예시일 뿐이다.*

input

`input([prompt])` 는 사용자 입력을 받는 함수이다. 입력 인수로 문자열을 전달하면 그 문자열은 프롬프트가 된다.

> *[]는 괄호 안의 내용을 생략할 수 있다는 관례 표기법이다.*

```
>>> a = input()
hi
>>> a
'hi'
>>> b = input("Enter: ")
Enter: hi
>>> b
'hi'
```

int

int(x)는 문자열 형태의 숫자나 소수점이 있는 숫자를 정수로 반환하는 함수이다. 정수가 입력되면 그대로 반환한다.

```
>>> int('3')
3
>>> int(3.4)
3
```

int(x, radix)는 radix 진수로 표현된 문자열 x를 10진수로 변환하여 반환한다. 예를 들어 2진수로 표현된 '11'의 10진수 값은 다음과 같이 구한다.

```
>>> int('11', 2)
3
```

16진수로 표현된 '1A'의 10진수 값은 다음과 같이 구한다.

```
>>> int('1A', 16)
26
```

isinstance

isinstance(object, class)는 첫 번째 인수로 객체, 두 번째 인수로 클래스를 받는다. 입력받은 객체가 해당 클래스의 인스턴스인지 판단하여 참이면 True, 거짓이면 False를 반환한다.

```
>>> class Person: pass
...
>>> a = Person()
>>> isinstance(a, Person)
True
```

위 예는 a가 Person 클래스로 만든 인스턴스임을 확인해 준다.

```
>>> b = 3
>>> isinstance(b, Person)
False
```

b는 Person 클래스로 만든 인스턴스가 아니므로 False를 반환한다.

len

len(s)는 입력값 s의 길이(요소의 전체 개수)를 반환하는 함수이다.

```
>>> len("python")
6
>>> len([1,2,3])
3
>>> len((1, 'a'))
2
```

list

list(iterable)는 반복 가능한 데이터를 입력받아 리스트로 만들어 반환하는 함수이다.

```
>>> list("python")
['p', 'y', 't', 'h', 'o', 'n']
>>> list((1,2,3))
[1, 2, 3]
```

list 함수에 리스트를 입력하면 똑같은 리스트를 복사하여 반환한다.

```
>>> a = [1, 2, 3]
>>> b = list(a)
>>> b
[1, 2, 3]
```

map

map(f, iterable)은 함수(f)와 반복 가능한 데이터를 입력으로 받는다. 입력받은 데이터의 각 요소에 함수 f를 적용한 결과를 반환하는 함수이다.

다음 예를 살펴보자.

```
# two_times.py
def two_times(numberList):
    result = []
    for number in numberList:
        result.append(number*2)
    return result

result = two_times([1, 2, 3, 4])
print(result)
```

two_times는 리스트를 입력받아 각 요소에 2를 곱해 반환하는 함수이다. 실행 결과는 다음과 같다.

```
[2, 4, 6, 8]
```

위 예제는 map 함수를 사용하여 다음처럼 바꿀 수 있다.

```
>>> def two_times(x):
...     return x*2
...
>>> list(map(two_times, [1, 2, 3, 4]))
[2, 4, 6, 8]
```

이 예제를 해석해 보자. 먼저 리스트의 첫 번째 요소인 1이 two_times 함수의 입력값으로 들어가 `1 * 2` 의 과정을 거쳐 2가 된다. 다음으로 리스트의 두 번째 요소인 2가 `2 * 2` 의 과정을 거쳐 4가 된다. 따라서 결괏값은 [2, 4]가 된다. 총 4개의 요솟값이 모두 처리되면 최종 결과는 [2, 4, 6, 8]이 된다. 이것이 map 함수가 하는 일이다.

> *map 함수의 결과를 리스트로 출력하기 위해 list 함수를 사용했다. map 함수는 map 객체를 반환한다.*

앞의 예는 lambda를 사용하여 다음처럼 간략하게 만들 수 있다.

```
>>> list(map(lambda a: a*2, [1, 2, 3, 4]))
[2, 4, 6, 8]
```

max

max(iterable)는 반복 가능한 데이터를 입력받아 최댓값을 반환하는 함수이다.

```
>>> max([1, 2, 3])
3
>>> max("python")
'y'
```

min

min(iterable)는 max 함수와 반대로, 반복 가능한 데이터를 입력받아 최솟값을 반환하는 함수이다.

```
>>> min([1, 2, 3])
1
>>> min("python")
'h'
```

oct

oct(x)는 정수를 8진수 문자열로 바꾸어 반환하는 함수이다.

```
>>> oct(34)
'0o42'
>>> oct(12345)
'0o30071'
```

open

`open(filename, [mode])` 은 '파일 이름'과 '읽기 방법'을 입력받아 파일 객체를 반환하는 함수이다. 읽기 방법(mode)을 생략하면 기본값인 읽기 모드(r)로 파일 객체를 만들어 반환한다.

mode	설명
w	쓰기 모드로 파일 열기
r	읽기 모드로 파일 열기
a	추가 모드로 파일 열기
b	바이너리 모드로 파일 열기

b는 w, r, a와 함께 사용한다. 예를 들어 rb는 '바이너리 읽기 모드'를 의미한다.

```
>>> f = open("binary_file", "rb")
```

ord

ord(c)는 문자의 유니코드 숫자 값을 반환하는 함수이다.

> *ord 함수는 chr 함수와 반대로 동작한다.*

```
>>> ord('a')
97
>>> ord('가')
44032
```

pow

pow(x, y)는 x를 y제곱한 결괏값을 반환하는 함수이다.

```
>>> pow(2, 4)
16
>>> pow(3, 3)
27
```

range

range([start,] stop [,step])은 for 문과 함께 자주 사용하는 함수이다. 입력받은 숫자에 해당하는 범위 값을 반복 가능한 객체로 만들어 반환한다.

인수가 하나일 경우

시작 숫자를 지정하지 않으면 range 함수는 0부터 시작한다.

```
>>> list(range(5))
[0, 1, 2, 3, 4]
```

인수가 2개일 경우

2개의 인수는 시작 숫자와 끝 숫자를 나타낸다. 단, 끝 숫자는 해당 범위에 포함되지 않는다.

```
>>> list(range(5, 10))
[5, 6, 7, 8, 9]
```

인수가 3개일 경우

세 번째 인수는 숫자 사이의 거리를 말한다.

```
>>> list(range(1, 10, 2))
[1, 3, 5, 7, 9]
>>> list(range(0, -10, -1))
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
```

round

`round(number [,ndigits])` 는 숫자를 입력받아 반올림해 반환하는 함수이다.

> *`[,ndigits]` 는 ndigits가 있을 수도 있고, 없을 수도 있다는 의미이다.*

```
>>> round(4.6)
5
>>> round(4.2)
4
```

다음과 같이 실수 5.678을 소수점 2자리까지만 반올림하여 표시할 수 있다.

```
>>> round(5.678, 2)
5.68
```

round 함수의 두 번째 인수는 반올림하여 표시할 소수점 자릿수(ndigits)를 의미한다.

sorted

sorted(iterable)는 입력 데이터를 정렬한 후 그 결과를 리스트로 반환하는 함수이다.

```
>>> sorted([3, 1, 2])
[1, 2, 3]
>>> sorted(['a', 'c', 'b'])
['a', 'b', 'c']
>>> sorted("zero")
['e', 'o', 'r', 'z']
>>> sorted((3, 2, 1))
[1, 2, 3]
```

리스트 자료형에도 sort 함수가 있다. 하지만 리스트의 sort 함수는 리스트 객체 자체를 정렬만 할 뿐, 정렬된 결과를 반환하지는 않는다.

str

str(object)는 객체를 문자열 형태로 변환하여 반환하는 함수이다.

```
>>> str(3)
'3'
>>> str('hi')
'hi'
```

sum

sum(iterable)은 입력 데이터의 합을 반환하는 함수이다.

```
>>> sum([1,2,3])
6
>>> sum((4,5,6))
15
```

tuple

tuple(iterable)은 반복 가능한 데이터를 튜플로 바꾸어 반환하는 함수이다. 입력이 튜플인 경우에는 그대로 반환한다.

```
>>> tuple("abc")
('a', 'b', 'c')
>>> tuple([1, 2, 3])
(1, 2, 3)
>>> tuple((1, 2, 3))
(1, 2, 3)
```

type

type(object)는 입력값의 자료형이 무엇인지 알려 주는 함수이다.

```
>>> type("abc")
<class 'str'>
>>> type([ ])
<class 'list'>
>>> type(open("test", 'w'))
<class '_io.TextIOWrapper'>
```

zip

`zip(*iterable)`은 동일한 개수로 이루어진 데이터들을 묶어서 반환하는 함수이다.

> *여기서 사용한 `*iterable`은 반복 가능한 데이터를 여러 개 입력할 수 있다는 의미이다.*

다음 예제로 사용법을 확인해 보자.

```
>>> list(zip([1, 2, 3], [4, 5, 6]))
[(1, 4), (2, 5), (3, 6)]
>>> list(zip([1, 2, 3], [4, 5, 6], [7, 8, 9]))
[(1, 4, 7), (2, 5, 8), (3, 6, 9)]
>>> list(zip("abc", "def"))
[('a', 'd'), ('b', 'e'), ('c', 'f')]
```

05-6 표준 라이브러리

이제 파이썬 프로그래밍 능력을 높여 줄 더 큰 날개를 달아 보자. 전 세계의 파이썬 고수들이 만든 유용한 프로그램을 모아 놓은 것이 바로 **파이썬 표준 라이브러리**이다. '라이브러리'는 '도서관'이라는 뜻 그대로 원하는 정보를 찾아보는 곳이다. 모든 라이브러리를 다 알 필요는 없고 어떤 일을 할 때 어떤 라이브러리를 사용해야 하는지 정도만 알면 된다. 이를 위해 어떤 라이브러리가 있고 어떻게 사용하는지 알아야 한다. 자주 사용하고 꼭 알아 두면 좋은 라이브러리를 중심으로 하나씩 살펴보자.

> - *파이썬 표준 라이브러리는 파이썬을 설치할 때 자동으로 컴퓨터에 설치된다.*
> - *sys, re 모듈은 파이썬의 중요한 표준 라이브러리이다. sys 모듈은 04-4, re 모듈은 08장 전체에 걸쳐 다루고 있으므로 이곳에서는 설명하지 않는다.*

- datetime.date
- time
  - time.time
  - time.localtime
  - time.asctime
  - time.ctime
  - time.strftime
  - time.sleep
- math.gcd
- math.lcm
- random
- itertools.zip_longest
- itertools.permutations
- itertools.combinations
- functools.reduce
- operator.itemgetter
- shutil
- glob
  - 디렉터리에 있는 파일들을 리스트로 만들기 - glob(pathname)
- pickle
- os
  - 내 시스템의 환경 변숫값을 알고 싶을 때 - os.environ
  - 디렉터리 위치 변경하기 - os.chdir

    - 디렉터리 위치 돌려받기 - os.getcwd
    - 시스템 명령어 호출하기 - os.system
    - 실행한 시스템 명령어의 결괏값 돌려받기 - os.popen
- zipfile
- threading
- tempfile
- traceback
- json
- urllib
- webbrowser

datetime.date

datetime.date는 연, 월, 일로 날짜를 표현할 때 사용하는 함수이다.

A 군과 B 양이 2021년 12월 14일부터 만나기 시작했다면 2023년 4월 5일은 둘이 사귄 지 며칠째 되는 날일까? 또한 사귀기 시작한 2021년 12월 14일은 무슨 요일이었을까? datetime.date 함수를 사용하면 이 문제를 쉽게 해결할 수 있다.



연, 월, 일로 다음과 같이 datetime.date 객체를 만들 수 있다.

```
>>> import datetime
>>> day1 = datetime.date(2021, 12, 14)
>>> day2 = datetime.date(2023, 4, 5)
```

이처럼 연, 월, 일을 인수로 하여 2021년 12월 14일에 해당하는 날짜 객체는 day1, 2023년 4월 5일에 해당하는 날짜 객체는 day2로 생성했다. 이렇게 날짜 객체를 만들었다면 두 날짜의 차이는 다음과 같이 뺄셈으로 쉽게 구할 수 있다.

```
>>> diff = day2 - day1
>>> diff.days
477
```

day2에서 day1을 빼면 datetime 모듈의 timedelta 객체가 반환된다. 이 객체를 diff 변수에 대입하고 diff.days를 이용하여 두 날짜의 차이를 확인했다.

요일은 datetime.date 객체의 weekday 함수를 사용하면 쉽게 구할 수 있다.

```
>>> day = datetime.date(2021, 12, 14)
>>> day.weekday()
1
```

0은 월요일을 의미하며 순서대로 1은 화요일, 2는 수요일, …, 6은 일요일이 된다. 월요일은 1, 화요일은 2, …, 일요일은 7을 반환하게 하려면 다음처럼 isoweekday 함수를 사용하면 된다.

```
>>> day.isoweekday()
2
```

2021년 12월 14일은 화요일이므로 isoweekday()를 사용하면 화요일을 뜻하는 2가 반환된다. weekday()를 사용하면 1이 반환된다.

time

시간과 관련된 time 모듈에는 함수가 매우 많다. 그중 가장 유용한 몇 가지만 알아보자.

time.time

time.time()은 UTC(universal time coordinated, 협정 세계 표준시)를 사용하여 현재 시간을 실수 형태로 반환하는 함수이다. 1970년 1월 1일 0시 0분 0초를 기준으로 지난 시간을 초 단위로 반환한다.

```
>>> import time
>>> time.time()
1684983953.5221913
```

time.localtime

time.localtime은 time.time()이 반환한 실숫값을 연, 월, 일, 시, 분, 초, ... 형태로 바꾸어 주는 함수이다.

```
>>> time.localtime(time.time())
time.struct_time(tm_year=2023, tm_mon=5, tm_mday=21, tm_hour=16,
    tm_min=48, tm_sec=42, tm_wday=1, tm_yday=141, tm_isdst=0)
```

time.asctime

time.asctime은 time.localtime이 반환한 튜플 형태의 값을 인수로 받아서 날짜와 시간을 알아보기 쉬운 형태로 반환하는 함수이다.

```
>>> time.asctime(time.localtime(time.time()))
'Fri Apr 28 20:50:20 2023'
```

time.ctime

`time.asctime(time.localtime(time.time()))` 은 간단하게 time.ctime()으로 표시할 수 있다. ctime이 asctime과 다른 점은 항상 현재 시간만을 반환한다는 점이다.

```
>>> time.ctime()
'Fri Apr 28 20:56:31 2023'
```

time.strftime

strftime 함수는 시간에 관계된 것을 세밀하게 표현하는 여러 가지 포맷 코드를 제공한다.

```
time.strftime('출력할 형식 포맷 코드', time.localtime(time.time()))
```

시간에 관계된 것을 표현하는 포맷 코드

포맷코드	설명	예
%a	요일의 줄임말	Mon
%A	요일	Monday
%b	달의 줄임말	Jan
%B	달	January
%c	날짜와 시간을 출력함.	Thu May 25 10:13:52 2023
%d	일(day)	[01,31]
%H	시간(hour): 24시간 출력 형태	[00,23]
%I	시간(hour): 12시간 출력 형태	[01,12]
%j	1년 중 누적 날짜	[001,366]
%m	달	[01,12]
%M	분	[00,59]
%p	AM or PM	AM
%S	초	[00,59]
%U	1년 중 누적 주(일요일 시작)	[00,53]
%w	숫자로 된 요일	[0(일), 6(토)]
%W	1년 중 누적 주(월요일 시작)	[00,53]
%x	현재 설정된 지역에 기반한 날짜 출력	05/25/23
%X	현재 설정된 지역에 기반한 시간 출력	17:22:21
%Y	연도 출력	2023
%Z	시간대 출력	대한민국 표준시

포맷코드	설명	예
%%	문자 %	%
%y	세기 부분을 제외한 연도 출력	01

다음은 time.strftime을 사용하는 예이다.

```
>>> import time
>>> time.strftime('%x', time.localtime(time.time()))
'05/25/23'
>>> time.strftime('%c', time.localtime(time.time()))
'Thu May 25 10:13:52 2023'
```

time.sleep

time.sleep 함수는 주로 루프 안에서 많이 사용한다. 이 함수를 사용하면 일정한 시간 간격을 두고 루프를 실행할 수 있다. 다음 예를 살펴보자.

```
# sleep1.py
import time
for i in range(10):
    print(i)
    time.sleep(1)
```

위 예는 1초 간격으로 0부터 9까지의 숫자를 출력한다. time.sleep 함수의 인수는 실수 형태를 쓸 수 있다. 즉 1이면 1초, 0.5면 0.5초가 되는 것이다.

점프 투 파이썬

인수 없이 time 함수 사용하기

time.localtime, time.asctime, time.strftime 함수는 다음처럼 입력 인수 없이 사용할 수 있다. 입력 인수 없이 사용하면 현재 시각을 기준으로 함수가 수행된다.

```
>>> time.localtime()
time.struct_time(tm_year=2023, tm_mon=6, tm_mday=18, tm_hour=17, tm_min=1, tm_sec=4, tm_wd
ay=6, tm_yday=169, tm_isdst=0)
>>> time.asctime()
'Sun Jun 18 17:01:09 2023'
>>> time.strftime('%c')
'Sun Jun 18 17:01:30 2023'
```

math.gcd

math.gcd 함수를 이용하면 최대 공약수(gcd, greatest common divisor)를 쉽게 구할 수 있다.

> - *math.gcd 함수는 파이썬 3.5 버전부터 사용할 수 있다.*
> - *공약수란 두 수 이상의 여러 수의 공통된 약수를 말하며 공약수 중 가장 큰 수를 최대 공약수라고 한다. 예를 들어 30과 15의 공약수는 1, 3, 5, 15이고, 최대 공약수는 15이다.*

어린이집에서 사탕 60개, 초콜릿 100개, 젤리 80개를 준비했다. 아이들이 서로 싸우지 않도록 똑같이 나누어 봉지에 담는다고 하면 최대 몇 봉지까지 만들 수 있을까? 단, 사탕, 초콜릿, 젤리는 남기지 않고 모두 담도록 한다.



이 문제는 60, 100, 80의 최대 공약수를 구하면 바로 해결된다. 똑같이 나눌 수 있는 봉지 개수가 최대가 되는 수를 구하면 된다.

```
>>> import math
>>> math.gcd(60, 100, 80)
20
```

> *파이썬 3.9 버전부터는 math.gcd에 여러 개의 인수를 입력할 수 있지만, 3.9 미만 버전에서는 2개까지만 허용된다.*

math.gcd() 함수로 최대 공약수를 구했더니 20이었다. 따라서 최대 20봉지를 만들 수 있다. 각 봉지에 들어가는 사탕, 초콜릿, 젤리의 개수는 다음과 같이 전체 개수를 최대 공약수 20으로 나누면 구할 수 있다.

```
>>> 60/20, 100/20, 80/20
(3.0, 5.0, 4.0)
```

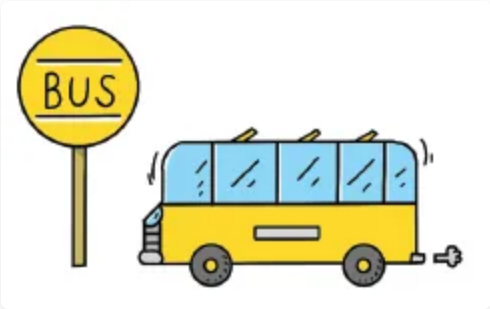
따라서 한 봉지당 사탕 3개씩, 초콜릿 5개씩, 젤리 4개씩 담으면 된다.

math.lcm

math.lcm은 최소 공배수(lcm, least common multiple)를 구할 때 사용하는 함수이다.

> - *math.lcm() 함수는 파이썬 3.9 버전부터 사용할 수 있다.*
> - *최소 공배수란 두 수의 공통 배수 중 가장 작은 수를 말한다. 예를 들어 3과 5의 최소 공배수는 15이다.*

어느 버스 정류장에 시내버스는 15분마다 도착하고 마을버스는 25분마다 도착한다고 한다. 오후 1시에 두 버스가 동시에 도착했다면 두 버스가 동시에 도착할 다음 시각을 알려면 어떻게 해야 할까?



이 문제는 15와 25의 공통 배수 중 가장 작은 수, 즉 최소 공배수를 구하면 바로 해결된다.

```
>>> import math
>>> math.lcm(15, 25)
75
```

math.lcm 함수를 사용하여 최소 공배수 75를 구했다. 따라서 두 버스가 동시에 도착할 다음 시각은 75분 후인 오후 2시 15분이다.

random

random은 난수(규칙이 없는 임의의 수)를 발생시키는 모듈이다. 먼저 random과 randint 함수에 대해 알아보자.

다음은 0.0에서 1.0 사이의 실수 중에서 난수 값을 반환하는 예이다.

```
>>> import random
>>> random.random()
0.53840103305098674
```

다음 예는 1에서 10 사이의 정수 중에서 난수 값을 반환한다.

```
>>> random.randint(1, 10)
6
```

다음 예는 1에서 55 사이의 정수 중에서 난수 값을 반환한다.

```
>>> random.randint(1, 55)
43
```

random 모듈을 사용해서 재미있는 함수를 하나 만들어 보자.

```
# random_pop.py
import random
def random_pop(data):
    number = random.randint(0, len(data)-1)
    return data.pop(number)

if __name__ == "__main__":
    data = [1, 2, 3, 4, 5]
    while data:
        print(random_pop(data))
```

[실행 결과]

```
2
3
1
5
4
```

random_pop 함수는 리스트의 요소 중에서 무작위로 하나를 선택하여 꺼낸 다음 그 값을 반환한다. 꺼낸 요소는 pop 메서드에 의해 사라진다.

random_pop 함수는 random 모듈의 choice 함수를 사용하여 다음과 같이 좀 더 직관적으로 만들 수도 있다.

```
def random_pop(data):
    number = random.choice(data)
    data.remove(number)
    return number
```

random.choice 함수는 입력으로 받은 리스트에서 무작위로 하나를 선택하여 반환한다.

리스트의 항목을 무작위로 섞고 싶을 때는 random.sample 함수를 사용하면 된다.

```
>>> import random
>>> data = [1, 2, 3, 4, 5]
>>> random.sample(data, len(data))
[5, 1, 3, 4, 2]
```

random.sample 함수에서 두 번째 인수인 `len(data)` 는 무작위로 추출할 원소의 개수를 의미한다.
`random.sample(data, 3)` 과 같이 사용하면 data 리스트에서 무작위로 3개를 추출하여 반환한다.

itertools.zip_longest

`itertools.zip_longest(*iterables, fillvalue=None)` 함수는 같은 개수의 자료형을 묶는 파이썬 내장 함수인 zip 함수와 똑같이 동작한다. 하지만 `itertools.zip_longest()` 함수는 전달한 반복 가능 객체(`*iterables`)의 길이가 서로 다르면 긴 객체의 길이에 맞춰 fillvalue에 설정한 값을 짧은 객체에 채울 수 있다.

예시로 유치원생 5명에게 간식을 나누어 주고자 다음과 같은 파이썬 코드를 작성해 보자.

```
# itertools_zip.py
students = ['한민서', '황지민', '이영철', '이광수', '김승민']
snacks = ['사탕', '초콜릿', '젤리']

result = zip(students, snacks)
print(list(result))
```

간식의 개수가 유치원생보다 적으므로 이 파이썬 코드를 실행하면 다음과 같은 결과가 나온다.

```
[('한민서', '사탕'), ('황지민', '초콜릿'), ('이영철', '젤리')]
```

students와 snacks의 요소 개수가 다르므로 더 적은 snacks의 개수만큼만 zip()으로 묶게 된다.

students의 요소 개수가 snacks보다 많을 때 그만큼을 '새우깡'으로 채우려면 어떻게 해야 할까? 이럴 때 요소 개수가 많은 것을 기준으로 자료형을 묶는 itertools.zip_longest()를 사용하면 된다. 부족한 항목은 None으로 채우는데, 다음처럼 fillvalue로 값을 지정하면 None 대신 다른 값으로 채울 수 있다.

```
# itertools_zip.py
import itertools

students = ['한민서', '황지민', '이영철', '이광수', '김승민']
snacks = ['사탕', '초콜릿', '젤리']

result = itertools.zip_longest(students, snacks, [[MARK]]fillvalue='새우깡'[[/MARK]])
print(list(result))
```

실행 결과는 다음과 같다.

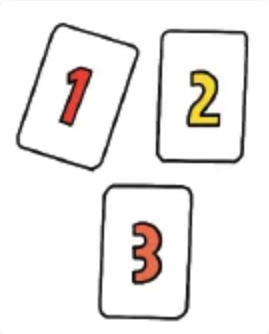
```
[('한민서', '사탕'), ('황지민', '초콜릿'), ('이영철', '젤리'), ('이광수', '새우깡'), ('김승민',
'새우깡')]
```

itertools.permutations

`itertools.permutations(iterable, r)`은 반복 가능 객체 중에서 r개를 선택한 순열을 이터레이터로 반환하는 함수이다.

> *이터레이터란 반복 가능한 객체를 의미한다.*

1, 2, 3이라는 숫자가 적힌 3장의 카드에서 2장의 카드를 꺼내 만들 수 있는 2자리 숫자를 모두 구하려면 어떻게 해야 할까?



[1, 2, 3]이라는 3장의 카드 중 순서에 상관없이 2장을 뽑는 경우의 수는 모두 3가지이다(조합).

- 1, 2
- 2, 3
- 1, 3

하지만 이 문제에서는 2자리 숫자이므로 이 3가지에 순서를 더해 다음처럼 6가지가 된다(순열).

- 1, 2
- 2, 1
- 2, 3
- 3, 2
- 1, 3
- 3, 1

이 순열은 itertools.permutations()를 사용하면 간단히 구할 수 있다.

```
>>> import itertools
>>> list(itertools.permutations(['1', '2', '3'], 2))
[('1', '2'), ('1', '3'), ('2', '1'), ('2', '3'), ('3', '1'), ('3', '2')]
```

따라서 만들 수 있는 2자리 숫자는 다음과 같이 모두 6가지이다.

```
>>> for a, b in itertools.permutations(['1', '2', '3'], 2):
...     print(a+b)
...
12
13
21
23
31
32
```

점프 투 파이썬

조합을 사용하는 함수

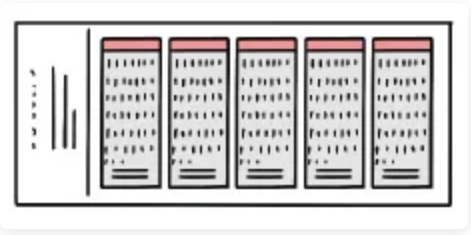
3장의 카드에서 순서에 상관없이 2장을 고르는 조합은 다음처럼 itertools.combinations()를 사용하면 된다.

```
>>> import itertools
>>> list(itertools.combinations(['1', '2', '3'], 2))
[('1', '2'), ('1', '3'), ('2', '3')]
```

itertools.combinations

`itertools.combinations(iterable, r)` 은 반복 가능 객체 중에서 r개를 선택한 조합을 이터레이터로 반환하는 함수이다.

1~45 중 서로 다른 숫자 6개를 뽑는 로또 번호의 모든 경우의 수(조합)를 구하고 그 개수를 출력하려면 어떻게 해야 할까?



다음과 같이 itertools.combinations()를 사용하면 45개의 숫자 중 6개를 선택하는 경우의 수를 구할 수 있다.

```
>>> import itertools
>>> it = itertools.combinations(range(1, 46), 6)
```

`itertools.combinations(range(1, 46), 6)`은 1~45의 숫자 중에서 6개를 뽑는 경우의 수를 이터레이터로 반환한다.

이터레이터 객체를 루프를 이용하여 출력하면 아마 끝도 없이 출력될 것이다. 궁금하다면 직접 실행해 봐도 좋다.

```
>>> for num in it:
...     print(num)
...
(1, 2, 3, 4, 5, 6)
(1, 2, 3, 4, 5, 7)
(1, 2, 3, 4, 5, 8)
(1, 2, 3, 4, 5, 9)
(1, 2, 3, 4, 5, 10)
(1, 2, 3, 4, 5, 11)
(1, 2, 3, 4, 5, 12)
(1, 2, 3, 4, 5, 13)
...
```

순환하여 출력하지 않고 이터레이터의 개수만 세려면 다음과 같이 하면 된다.

```
>>> len(list(itertools.combinations(range(1, 46), 6)))
8145060
```

선택할 수 있는 로또 번호의 가짓수는 8,145,060이다.

> *반드시 로또에 당첨되길 희망한다면 서로 다른 번호로 구성한 8,145,060장의 로또를 사면 된다. 게임 1번에 1,000원이라고 할 때 그 금액은 무려 81억 4,506만 원이다.*

점프 투 파이썬

중복 조합을 사용하는 함수

로또 복권이 숫자 중복을 허용하도록 규칙이 변경된다면 경우의 수는 몇 개가 될까?

> *중복이 허용된다는 것은 당첨 번호가 [1, 2, 3, 4, 5, 5]처럼 5가 2번 이상 나와도 되고 [1, 1, 1, 1, 1, 1]처럼 1이 6번 나와도 된다는 의미이다.*

같은 숫자를 허용하는 중복 조합은 itertools.combinations_with_replacement()를 사용하면 된다.

```
>>> len(list(itertools.combinations_with_replacement(range(1, 46), 6)))
15890700
```

당연히 중복을 허용하지 않을 때보다 훨씬 많은 경우의 수가 반환되는 것을 확인할 수 있다.

functools.reduce

`functools.reduce(function, iterable)`은 함수(function)를 반복 가능한 객체(iterable)의 요소에 차례대로(왼쪽에서 오른쪽으로) 누적 적용하여 이 객체를 하나의 값으로 줄이는 함수이다.

다음은 입력 인수 data의 요소를 모두 더하여 반환하는 add 함수이다.

```
def add(data):
    result = 0
    for i in data:
        result += i
    return result

data = [1, 2, 3, 4, 5]
result = add(data)
print(result)  # 15 출력
```

functools.reduce()를 사용하여 같은 동작을 하는 코드를 작성하려면 어떻게 해야 할까? functools.reduce()를 사용한 코드는 다음과 같다.

```
# reduce_test.py
import functools

data = [1, 2, 3, 4, 5]
result = functools.reduce(lambda x, y: x + y, data)
print(result)  # 15 출력
```

functools.reduce()를 사용하면 reduce()에 선언한 람다 함수를 data 요소에 차례대로 누적 적용하여 다음과 같이 계산한다.

```
((((1+2)+3)+4)+5)
```

따라서 앞서 본 add 함수와 동일한 역할을 하게 된다.

점프 투 파이썬

functools.reduce()로 최댓값 구하기

functools.reduce 함수로 최댓값도 구할 수 있다.

```
num_list = [3, 2, 8, 1, 6, 7]
max_num = functools.reduce(lambda x, y: x if x > y else y, num_list)
print(max_num)  # 8 출력
```

[3, 2, 8, 1, 6, 7] 요소를 차례대로 reduce()의 람다 함수로 전달하여 두 값 중 큰 값을 선택하고 마지막에 남은 최댓값을 반환한다.

> *최솟값은 `functools.reduce(lambda x, y: x if x < y else y, num_list)` 로 구할 수 있다.*

operator.itemgetter

operator.itemgetter는 주로 sorted와 같은 함수의 key 매개변수에 적용하여 다양한 기준으로 정렬할 수 있도록 도와주는 모듈이다.

예를 들어 학생의 이름, 나이, 성적 등의 정보를 저장한 다음과 같은 students 리스트가 있다고 가정해 보자.

```
students = [
    ("jane", 22, 'A'),
    ("dave", 32, 'B'),
    ("sally", 17, 'B'),
]
```

students 리스트에는 3개의 튜플이 있으며 각 튜플은 순서대로 이름, 나이, 성적에 해당하는 데이터로 이루어졌다. 이 리스트를 나이순으로 정렬하려면 어떻게 해야 할까?



이 문제는 다음처럼 sorted 함수의 key 매개변수에 itemgetter()를 적용하면 쉽게 해결할 수 있다.

```
# itemgetter1.py
from operator import itemgetter

students = [
    ("jane", 22, 'A'),
    ("dave", 32, 'B'),
    ("sally", 17, 'B'),
]

result = sorted(students, key=itemgetter(1))
print(result)
```

이 파일을 실행하여 출력해 보면 다음과 같이 나이 순서대로 정렬한 것을 확인할 수 있다.

```
[('sally', 17, 'B'), ('jane', 22, 'A'), ('dave', 32, 'B')]
```

itemgetter(1)은 students의 아이템인 튜플의 두 번째 요소를 기준으로 정렬하겠다는 의미이다. itemgetter(2)와 같이 사용하면 성적순으로 정렬한다.

이번에는 students의 요소가 다음처럼 딕셔너리일 때를 생각해 보자.

```
students = [
    {"name": "jane", "age": 22, "grade": 'A'},
    {"name": "dave", "age": 32, "grade": 'B'},
    {"name": "sally", "age": 17, "grade": 'B'},
]
```

딕셔너리일 때도 마찬가지로 age를 기준으로 정렬해 보자. 이때도 itemgetter()를 적용하면 된다. 단, 이번에는 `itemgetter('age')` 처럼 딕셔너리의 키를 사용해야 한다. `itemgetter('age')` 는 딕셔너리의 키인 age를 기준으로 정렬

하겠다는 의미이다.

```
# itemgetter2.py
from operator import itemgetter

students = [
    {"name": "jane", "age": 22, "grade": 'A'},
    {"name": "dave", "age": 32, "grade": 'B'},
    {"name": "sally", "age": 17, "grade": 'B'},
]

result = sorted(students, [[MARK]]key=itemgetter('age')[[/MARK]])
print(result)
```

출력 결과는 다음과 같이 age 순으로 정렬된 것을 확인할 수 있다.

```
[{'name': 'sally', 'age': 17, 'grade': 'B'}, {'name': 'jane', 'age': 22, 'grade': 'A'}, {'nam
e': 'dave', 'age': 32, 'grade': 'B'}]
```

점프 투 파이썬

operator.attrgetter()

students 리스트의 요소가 튜플이 아닌 Student 클래스의 객체라면 다음처럼 attrgetter()를 적용하여 정렬해야 한다.

```
# attrgetter1.py
from operator import attrgetter

class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

students = [
    Student('jane', 22, 'A'),
    Student('dave', 32, 'B'),
    Student('sally', 17, 'B'),
]

result = sorted(students, [[MARK]]key=attrgetter('age')[[/MARK]])
```

`attrgetter('age')` 는 Student 객체의 age 속성으로 정렬하겠다는 의미이다. `attrgetter('grade')` 와 같이 사용하면 성적순으로 정렬한다.

shutil

shutil은 파일을 복사(copy)하거나 이동(move)할 때 사용하는 모듈이다.

작업 중인 파일을 자동으로 백업하는 기능을 구현하고자 `c:\doit\a.txt` 를 `c:\temp\a.txt.bak` 이라는 이름으로 복사하는 프로그램을 만들고자 한다. 어떻게 만들어야 할까? `c:\doit` 디렉터리에 a.txt를 만드는 중이며 백업용 `c:\temp` 디렉터리는 이미 만들었다고 가정한다.

다음은 shutil을 사용한 방법이다.

```
# shutil_copy.py
import shutil

shutil.copy("c:/doit/a.txt", "c:/temp/a.txt.bak")
```

점프 투 파이썬

shutil.move로 삭제 기능 만들기

휴지통으로 삭제하는 기능을 구현하고자 `c:\doit\a.txt` 파일을 `c:\temp\a.txt`로 이동하려면 다음과 같이 코드를 작성해야 한다.

```
# shutil_move.py
import shutil

shutil.move("c:/doit/a.txt", "c:/temp/a.txt")
```

glob

가끔 파일을 읽고 쓰는 기능이 있는 프로그램을 만들다 보면 특정 디렉터리에 있는 파일 이름 모두를 알아야 할 때가 있다. 이럴 때 사용하는 모듈이 바로 glob이다.

디렉터리에 있는 파일들을 리스트로 만들기 - glob(pathname)

glob 모듈은 디렉터리 안의 파일들을 읽어서 반환한다. `*`, `?` 등 메타 문자를 써서 원하는 파일만 읽어 들일 수도 있다. 다음은 `C:/doit` 디렉터리에 있는 파일 중 이름이 mark로 시작하는 파일을 모두 찾아서 읽어들이는 예이다.

> *`?`는 1자리 문자열, `*`은 임의의 길이의 문자열을 의미한다.*

```
>>> import glob
>>> glob.glob("c:/doit/mark*")
['c:/doit\\marks1.py', 'c:/doit\\marks2.py', 'c:/doit\\marks3.py']
>>>
```

pickle

pickle은 객체의 형태를 그대로 유지하면서 파일에 저장하고 불러올 수 있게 하는 모듈이다. 다음 예는 pickle 모듈의 dump 함수를 사용하여 딕셔너리 객체인 data를 그대로 파일에 저장하는 방법을 보여 준다.

```
>>> import pickle
>>> f = open("test.txt", 'wb')
>>> data = {1: 'python', 2: 'you need'}
>>> pickle.dump(data, f)
>>> f.close()
```

다음은 pickle.dump로 저장한 파일을 pickle.load를 사용해서 원래 있던 딕셔너리 객체(data) 상태 그대로 불러오는 예이다.

```
>>> import pickle
>>> f = open("test.txt", 'rb')
>>> data = pickle.load(f)
>>> print(data)
{2:'you need', 1:'python'}
```

위 예에서는 딕셔너리 객체를 사용했지만, 어떤 자료형이든 저장하고 불러올 수 있다.

os

os 모듈은 환경 변수나 디렉터리, 파일 등의 OS 자원을 제어할 수 있게 해 주는 모듈이다.

내 시스템의 환경 변숫값을 알고 싶을 때 - os.environ

시스템은 제각기 다른 환경 변숫값을 가지고 있는데, os.environ은 현재 시스템의 환경 변숫값을 반환한다. 다음을 따라 해 보자.

```
>>> import os
>>> os.environ
environ({'PROGRAMFILES': 'C:\\Program Files', 'APPDATA': … 생략 …})
```

이 결괏값은 필자의 시스템 정보이다. os.environ은 환경 변수에 대한 정보를 딕셔너리 형태로 구성된 environ 객체로 반환한다. 자세히 보면 여러 가지 유용한 정보를 찾을 수 있다.

반환받은 객체는 다음과 같이 호출하여 사용할 수 있다. 다음은 필자 시스템의 PATH 환경 변수 내용이다.

```
>>> os.environ['PATH']
'C:\\ProgramData\\Oracle\\Java\\javapath; ... 생략 ...'
```

디렉터리 위치 변경하기 - os.chdir

os.chdir를 사용하면 다음과 같이 현재 디렉터리의 위치를 변경할 수 있다.

```
>>> os.chdir("C:\\WINDOWS")
```

디렉터리 위치 돌려받기 - os.getcwd

os.getcwd는 현재 자신의 디렉터리 위치를 반환한다.

```
>>> os.getcwd()
'C:\WINDOWS'
```

시스템 명령어 호출하기 - os.system

시스템 자체의 프로그램이나 기타 명령어를 파이썬에서 호출할 수도 있다. `os.system("명령어")` 처럼 사용한다. 다음은 현재 디렉터리에서 시스템 명령어 dir을 실행하는 예이다.

```
>>> os.system("dir")
```

실행한 시스템 명령어의 결괏값 돌려받기 - os.popen

os.popen은 시스템 명령어를 실행한 결괏값을 읽기 모드 형태의 파일 객체로 반환한다.

```
>>> f = os.popen("dir")
```

읽어 들인 파일 객체의 내용을 보려면 다음과 같이 하면 된다.

```
>>> print(f.read())
```

이 밖에 유용한 os 관련 함수는 다음과 같다.

함수	설명
os.mkdir(디렉터리)	디렉터리를 생성한다.
os.rmdir(디렉터리)	디렉터리를 삭제한다. 단, 디렉터리가 비어 있어야 삭제할 수 있다.
os.remove(파일)	파일을 지운다.
os.rename(src, dst)	src라는 이름의 파일을 dst라는 이름으로 바꾼다.

zipfile

zipfile은 여러 개의 파일을 zip 형식으로 합치거나 이를 해제할 때 사용하는 모듈이다.

다음과 같은 3개의 텍스트 파일이 있다고 가정해 보자.

```
a.txt
b.txt
c.txt
```

이 3개의 텍스트 파일을 하나로 합쳐 'mytext.zip'이라는 파일을 만들고 이 파일을 원래의 텍스트 파일 3개로 해제하는 프로그램을 만들려면 어떻게 해야 할까?

zipfile.ZipFile()을 사용하여 해결해 보자.

```
# zipfile_test.py
import zipfile

# 파일 합치기
with zipfile.ZipFile('mytext.zip', 'w') as myzip:
    myzip.write('a.txt')
    myzip.write('b.txt')
    myzip.write('c.txt')

# 해제하기
with zipfile.ZipFile('mytext.zip') as myzip:
    myzip.extractall()
```

ZipFile 객체의 write() 함수로 개별 파일을 추가할 수 있고 extractall() 함수를 사용하면 모든 파일을 해제할 수 있다.

합친 파일에서 특정 파일만 해제하고 싶다면 다음과 같이 extract() 함수를 사용하면 된다.

```
# 특정 파일만 해제하기
with zipfile.ZipFile('mytext.zip') as myzip:
    myzip.extract('a.txt')
```

파일을 압축하여 묶고 싶은 경우에는 compression, compresslevel 옵션을 사용할 수 있다.

```
# 압축하여 묶기
with zipfile.ZipFile('mytext.zip', 'w', [[MARK]]compression=zipfile.ZIP_LZMA, compresslevel=9
[[/MARK]]) as myzip:
    ( ... 생략 ...)
```

compression에는 4가지 종류가 있다.

- ZIP_STORED: 압축하지 않고 파일을 zip으로만 묶는다. 속도가 빠르다.
- ZIP_DEFLATED: 일반적인 zip 압축으로 속도가 빠르고 압축률은 낮다(호환성이 좋다).
- ZIP_BZIP2: bzip2 압축으로 압축률이 높고 속도가 느리다.
- ZIP_LZMA: lzma 압축으로 압축률이 높고 속도가 느리다(7zip과 동일한 알고리즘으로 알려져 있다).

compresslevel은 압축 수준을 의미하는 숫자값으로, 1~9를 사용한다. 1은 속도가 가장 빠르지만 압축률이 낮고, 9는 속도는 가장 느리지만 압축률이 높다.

threading

스레드 프로그래밍은 초보 프로그래머가 구현하기에는 매우 어려운 기술이다. 여기에 잠시 소개했으므로 눈으로만 살펴보고 넘어가자.

컴퓨터에서 동작하고 있는 프로그램을 프로세스(process)라고 한다. 보통 1개의 프로세스는 1가지 일만 하지만, 스레드(thread)를 사용하면 한 프로세스 안에서 2가지 또는 그 이상의 일을 동시에 수행할 수 있다.

간단한 예제로 설명을 대신하겠다.

```
# thread_test.py
import time

def long_task():  # 5초의 시간이 걸리는 함수
    for i in range(5):
        time.sleep(1)  # 1초간 대기한다.
        print(f"working:{i}\n")

print("Start")

for i in range(5):  # long_task를 5회 수행한다.
    long_task()

print("End")
```

long_task는 수행하는 데 5초의 시간이 걸리는 함수이다. 위 프로그램은 이 함수를 총 5번 반복해서 수행하는 프로그램이다. 이 프로그램은 5초가 5번 반복되므로 총 25초의 시간이 걸린다.

하지만 앞에서 설명했듯이 스레드를 사용하면 5초가 걸리는 long_task 함수를 동시에 실행할 수 있으므로 시간을 줄일 수 있다.

다음과 같이 프로그램을 수정해 보자.

```
# thread_test.py
import time
import threading  # 스레드를 생성하기 위해서는 threading 모듈이 필요하다.

def long_task():
    for i in range(5):
        time.sleep(1)
        print(f"working:{i}\n")

print("Start")

threads = []
for i in range(5):
    t = threading.Thread(target=long_task)  # 스레드를 생성한다.
    threads.append(t)

for t in threads:
    t.start()  # 스레드를 실행한다.

print("End")
```

이와 같이 프로그램을 수정하고 실행해 보면 25초 걸리던 작업이 5초 정도에 수행되는 것을 확인할 수 있다.
threading.Thread를 사용하여 만든 스레드 객체가 동시 작업을 가능하게 해 주기 때문이다.

하지만 프로그램을 실행해 보면 "Start"와 "End"가 먼저 출력되고 그 이후에 스레드의 결과가 출력되는 것을 확인할 수 있다. 그리고 프로그램이 정상 종료되지 않는다. 우리가 기대하는 것은 "Start"가 출력되고 그다음에 스레드의 결과가 출력된 후 마지막으로 "End"가 출력되는 것이다.

이 문제를 해결하려면 프로그램을 다음과 같이 수정해야 한다.

```
# thread_test.py
import time
import threading

def long_task():
    for i in range(5):
        time.sleep(1)
        print(f"working:{i}\n")

print("Start")

threads = []
for i in range(5):
    t = threading.Thread(target=long_task)
    threads.append(t)

for t in threads:
    t.start()

[[MARK]]for t in threads:
    t.join()  # join으로 스레드가 종료될때까지 기다린다.[[/MARK]]

print("End")
```

스레드의 join 함수는 해당 스레드가 종료될 때까지 기다리게 한다. 따라서 위와 같이 수정하면 우리가 원하던 출력을 보게 된다.

tempfile

파일을 임시로 만들어서 사용할 때 유용한 모듈이 바로 tempfile이다. `tempfile.mkstemp()`는 중복되지 않는 임시 파일을 만들고 `(파일 디스크립터, 파일 경로)` 형태의 튜플을 반환한다.

```
>>> import tempfile
>>> fd, path = tempfile.mkstemp()
>>> path
'C:\\WINDOWS\\TEMP\\~-275151-0'
```

`tempfile.TemporaryFile()`은 임시 저장 공간으로 사용할 파일 객체를 반환한다. 이 파일은 기본적으로 바이너리 쓰기 모드(wb)를 갖는다. `f.close()`가 호출되면 이 파일은 자동으로 삭제된다.

```
>>> import tempfile
>>> f = tempfile.TemporaryFile()
>>> f.close()
```

traceback

traceback은 프로그램 실행 중 발생한 오류를 추적하고자 할 때 사용하는 모듈이다.

다음과 같은 코드를 작성하여 실행해 보자.

```
# traceback_test.py
def a():
    return 1/0

def b():
    a()

def main():
    try:
        b()
    except:
        print("오류가 발생했습니다.")

main()
```

프로그램 실행 결과는 다음과 같다.

```
오류가 발생했습니다.
```

main() 함수가 시작되면 b() 함수를 호출하고 b() 함수에서 다시 a() 함수를 호출하여 1을 0으로 나누므로 오류가 발생하여 "오류가 발생했습니다."라는 메시지를 출력했다.

> *이렇게 간단한 프로그램이 아니라 복잡한 파이썬 코드라면 어디에서 어떤 오류가 발생하는지 알기 어렵다.*

이때 이 코드에서 오류가 발생한 위치와 원인을 정확히 판단할 수 있도록 코드를 업그레이드하려면 어떻게 해야 할까?

오류가 발생한 위치에 다음과 같이 traceback 모듈을 적용해 보자.

```
# traceback_test.py
[[MARK]]import traceback[[/MARK]]

def a():
    return 1/0

def b():
    a()

def main():
    try:
        b()
    except:
        print("오류가 발생했습니다.")
        [[MARK]]print(traceback.format_exc())[[/MARK]]

main()
```

오류가 발생한 위치에 `print(traceback.format_exc())` 문장을 추가했다. traceback 모듈의 format_exc()는 오류 추적 결과를 문자열로 반환하는 함수이다. 이렇게 코드를 수정하고 다시 프로그램을 실행하면 다음과 같이 출력될 것이다.

```
오류가 발생했습니다.
Traceback (most recent call last):
  File "c:\doit\traceback_sample.py", line 14, in main
    b()
  File "c:\doit\traceback_sample.py", line 9, in b
    a()
  File "c:\doit\traceback_sample.py", line 5, in a
    return 1/0
ZeroDivisionError: division by zero
```

오류 추적을 통해 main() 함수에서 b() 함수를 호출하고 b() 함수에서 다시 a() 함수를 호출하여 `1 / 0`을 실행하려 하므로 0으로 나눌 수 없다는 ZeroDivisionError가 발생했다는 것을 로그를 통해 정확하게 확인할 수 있다.

json

json은 JSON 데이터를 쉽게 처리하고자 사용하는 모듈이다.

다음은 개인정보를 JSON 형태의 데이터로 만든 myinfo.json 파일이다.

`[파일명: myinfo.json]`

```
{
    "name": "홍길동",
    "birth": "0525",
    "age": 30
}
```

인터넷으로 얻은 이 파일을 읽어 파이썬에서 처리할 수 있도록 딕셔너리 자료형으로 만들려면 어떻게 해야 할까?

JSON 파일을 읽어 딕셔너리로 변환하려면 다음처럼 json 모듈을 사용하면 된다.

```
>>> import json
>>> with open('myinfo.json') as f:
...     data = json.load(f)
...
>>> type(data)
<class 'dict'>
>>> data
{'name': '홍길동', 'birth': '0525', 'age': 30}
```

JSON 파일을 읽을 때는 이 예처럼 `json.load(파일_객체)`를 사용한다. load() 함수는 읽은 데이터를 딕셔너리 자료형으로 반환한다. 반대로 딕셔너리 자료형을 JSON 파일로 생성할 때는 다음처럼 `json.dump(딕셔너리, 파일_객체)`를 사용한다.

```
>>> import json
>>> data = {'name': '홍길동', 'birth': '0525', 'age': 30}
>>> with open('myinfo.json', 'w') as f:
...     json.dump(data, f)
```

이번에는 파이썬 자료형을 JSON 문자열로 만드는 방법에 대해서 알아보자.

```
>>> import json
>>> d = {"name":"홍길동", "birth":"0525", "age": 30}
>>> json_data = json.dumps(d)
>>> json_data
'{"name": "\\ud64d\\uae38\\ub3d9", "birth": "0525", "age": 30}'
```

딕셔너리 자료형을 JSON 문자열로 만들려면 json.dumps() 함수를 사용하면 된다. 그런데 딕셔너리를 JSON 데이터로 변경하면 '홍길동'과 같은 한글 문자열이 코드 형태로 표시된다. dump(), dumps() 함수는 기본적으로 데이터를 아스키 형태로 저장하기 때문이다. 유니코드 문자열을 아스키 형태로 저장하다 보니 한글 문자열이 마치 깨진 것처럼 보인다.

그러나 JSON 문자열을 딕셔너리로 다시 역변환하여 사용하는 데는 전혀 문제가 없다. JSON 문자열을 딕셔너리로 변환할 때는 다음처럼 json.loads() 함수를 사용한다.

```
>>> json.loads(json_data)
{'name': '홍길동', 'birth': '0525', 'age': 30}
```

한글 문자열이 아스키 형태의 문자열로 변경되는 것을 방지하는 방법도 있다.

```
>>> d = {"name":"홍길동", "birth":"0525", "age": 30}
>>> json_data = json.dumps(d, ensure_ascii=False)
>>> json_data
'{"name": "홍길동", "birth": "0525", "age": 30}'
>>> json.loads(json_data)
{'name': '홍길동', 'birth': '0525', 'age': 30}
```

이처럼 `ensure_ascii=False` 옵션을 사용하면 된다. 이 옵션은 데이터를 저장할 때 아스키 형태로 변환하지 않겠다는 뜻이다.

출력되는 JSON 문자열을 보기 좋게 정렬하려면 다음처럼 indent 옵션을 추가하면 된다.

```
>>> d = {"name":"홍길동", "birth":"0525", "age": 30}
>>> print(json.dumps(d, indent=2, ensure_ascii=False))
{
  "name": "홍길동",
  "birth": "0525",
  "age": 30
}
```

딕셔너리 외에 리스트나 튜플처럼 다른 자료형도 JSON 문자열로 바꿀 수 있다.

```
>>> json.dumps([1,2,3])
'[1, 2, 3]'
>>> json.dumps((4,5,6))
'[4, 5, 6]'
```

urllib

urllib은 URL을 읽고 분석할 때 사용하는 모듈이다.

브라우저로 위키독스의 특정 페이지를 읽으려면 다음과 같이 요청하면 된다.

```
https://wikidocs.net/페이지_번호 (예: https://wikidocs.net/12)
```

그러면 오프라인으로도 읽을 수 있도록 페이지 번호를 입력받아 위키독스의 특정 페이지를 `wikidocs_페이지_번호.html` 파일로 저장하는 함수는 어떻게 만들어야 할까?

URL을 호출하여 원하는 리소스를 얻으려면 urllib 모듈을 사용해야 한다.

```
# urllib_test.py
import urllib.request

def get_wikidocs(page):
    resource = 'https://wikidocs.net/{}'.format(page)
    with urllib.request.urlopen(resource) as s:
        with open('wikidocs_%s.html' % page, 'wb') as f:
            f.write(s.read())
```

get_wikidocs(page)는 위키독스의 페이지 번호를 입력받아 해당 페이지의 리소스 내용을 파일로 저장하는 함수이다. 이 코드에서 보듯이 `urllib.request.urlopen(resource)`로 s 객체를 생성하고 `s.read()`로 리소스 내용 전체를 읽어 이를 저장할 수 있다. 예를 들어 `get_wikidocs(12)`라고 호출하면 `https://wikidocs.net/12` 웹 페이지를 `wikidocs_12.html`라는 파일로 저장한다.

점프 투 파이썬

SSL 오류가 발생하면

get_wikidocs 함수 실행 시 다음과 같은 오류가 발생할 수 있다.

```
urllib.error.URLError: <urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed
```

이 오류는 SSL 인증서 검증에 실패했음을 나타낸다. 파이썬이 서버의 SSL 인증서를 신뢰할 수 있는지 확인하는 과정에서, 로컬 시스템에 필요한 CA(인증 기관) 인증서가 없거나 찾을 수 없어 발생한 문제로 특히 macOS에서 자주 발생한다.

이럴 때는 먼저 로컬 인증서(신뢰할 수 있는 CA 인증서)를 올바르게 설치하는 방법을 우선 검토해야 한다. SSL 인증서 검증 비활성화는 보안상 위험하므로 권장되지 않는다.

부득이하게 개발 또는 테스트 환경에서만 일시적으로 확인이 필요할 때는 다음처럼 사용하고, 실제 서비스 코드에서는 사용하지 말자.

```
import urllib.request

# 개발/테스트용 임시 코드: SSL 인증서 검증 비활성화(운영 환경 사용 금지)
import ssl
context = ssl._create_unverified_context()

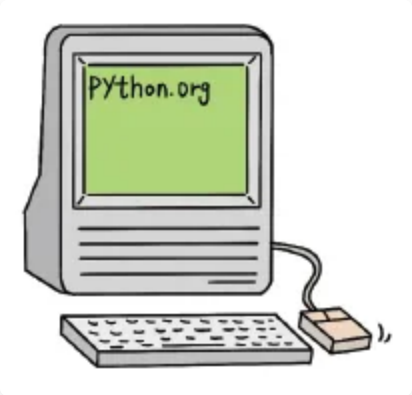
def get_wikidocs(page):
    resource = f'https://wikidocs.net/{page}'
    with urllib.request.urlopen(resource, context=context) as s:
        with open(f'wikidocs_{page}.html', 'wb') as f:
            f.write(s.read())

get_wikidocs(12)
```

webbrowser

webbrowser는 파이썬 프로그램에서 시스템 브라우저를 호출할 때 사용하는 모듈이다.

개발 중 궁금한 내용이 있어 파이썬 문서를 참고하려 한다. 이를 위해 https://python.org 사이트를 새로운 웹 브라우저로 열려면 코드를 어떻게 작성해야 할까?



파이썬으로 웹 페이지를 새 창으로 열려면 webbrowser 모듈의 open_new() 함수를 사용해야 한다.

```
# webbrowser_test.py
import webbrowser

webbrowser.open_new('http://python.org')
```

이미 열린 브라우저로 원하는 사이트를 열고 싶다면 다음처럼 open_new() 대신 open()을 사용하면 된다.

```
webbrowser.open('http://python.org')
```

> *파이썬 라이브러리에 대해서 보다 깊게 공부하고 싶다면 필자가 집필한 《점프 투 파이썬 - 라이브러리 예제 편》을 읽어 보기 바란다.*

05-7 외부 라이브러리

파이썬 설치 시 기본으로 설치되는 라이브러리를 '파이썬 표준 라이브러리'라고 한다. 이번에 소개하는 외부 라이브러리는 파이썬 표준 라이브러리가 아니므로 사용하려면 먼저 설치 도구를 이용해 설치해야 한다.

> *pip은 '핍'이라고 읽는다.*

- pip
  - pip install
  - pip uninstall
  - 특정 버전으로 설치하기
  - 최신 버전으로 업그레이드하기
  - 설치된 패키지 확인하기
- Faker
  - Faker 사용해 보기
  - Faker 활용하기
- sympy
  - sympy 사용해 보기
  - sympy 활용

pip

pip은 파이썬 모듈이나 패키지를 쉽게 설치할 수 있도록 도와주는 도구이다. pip의 가장 큰 장점은 의존성 있는 모듈이나 패키지를 자동으로 함께 설치해 준다는 것이다. 예를 들어 B 패키지를 설치하려면 A 패키지가 먼저 필요하다면, pip은 B 패키지를 설치할 때 A 패키지도 자동으로 함께 설치해 준다.

pip 사용법을 간략하게 알아보자.

pip install

PyPI(Python Package Index)는 파이썬 소프트웨어가 모인 저장 공간이다. 많은 파이썬 패키지가 등록되어 있으며, 누구나 내려받아 사용할 수 있다. 직접 내려받아 설치해도 되지만, pip을 이용하면 다음과 같이 간편하게 설치할 수 있다.

```
python -m pip install SomePackage
```

> *맥에서도 동일하게 `python3 -m pip install SomePackage` 또는 `python -m pip install SomePackage` 형태로 실행할 수 있다.*

여기서 SomePackage는 설치할 패키지 이름이다.

pip uninstall

설치한 패키지를 삭제하려면 다음 명령어를 사용한다.

```
python -m pip uninstall SomePackage
```

특정 버전으로 설치하기

특정 버전의 패키지를 설치하려면 다음과 같이 버전을 지정한다. 아래 명령어를 실행하면 1.0.4 버전의 SomePackage를 설치한다.

```
python -m pip install SomePackage==1.0.4
```

버전을 생략하면 최신 버전으로 자동 설치된다.

```
python -m pip install SomePackage
```

최신 버전으로 업그레이드하기

패키지를 최신 버전으로 업그레이드하려면 `--upgrade` 옵션을 사용한다.

```
python -m pip install --upgrade SomePackage
```

설치된 패키지 확인하기

다음 명령은 pip으로 설치한 패키지 목록을 출력한다.

```
python -m pip list
```

다음과 같이 설치된 패키지 목록을 출력한다.

```
Package                   Version
------------------------- -------
amqp                      2.1.4
anyjson                   0.3.3
billiard                  3.3.0.23
celery                    3.1.0
defusedxml                0.4.1
diff-match-patch          20121119
(... 생략 ...)
```

Faker

이번에는 pip을 사용하여 유용한 외부 라이브러리 중 하나인 Faker를 설치하고 사용해 보자. Faker는 테스트용 가짜 데이터를 생성할 때 사용하는 라이브러리이다.

Faker 라이브러리는 pip을 이용하여 설치해야 한다.

```
C:\> python -m pip install Faker
```

Faker 사용해 보기

다음과 같은 형식의 테스트 데이터 30건이 필요하다고 가정해 보자. 직접 데이터를 작성하지 않고 좀 더 편리하게 테스트 데이터를 만들려면 어떻게 해야 할까?

```
[(이름1, 주소1), (이름2, 주소2), ..., (이름30, 주소30)]
```

Faker를 사용하면 매우 쉽게 만들 수 있다. 이름은 다음처럼 만든다.

```
>>> from faker import Faker
>>> fake = Faker()
>>> fake.name()
'Matthew Estrada'
```

한글 이름이 필요하다면 다음과 같이 한국을 의미하는 ko-KR을 전달하여 fake 객체를 생성하면 된다.

```
>>> fake = Faker('ko-KR')
>>> fake.name()
'김하은'
```

주소는 다음과 같이 만든다.

```
>>> fake.address()
'충청북도 수원시 잠실6길 (경자주이읍)'
```

따라서 이름과 주소를 쌍으로 하는 30건의 테스트 데이터는 다음과 같이 만들 수 있다.

```
>>> test_data = [(fake.name(), fake.address()) for i in range(30)]
```

실행 결과는 다음과 같다.

```
>>> test_data
[('이예진', '인천광역시 동대문구 언주거리 (경자김면)'), ('윤도윤', '광주광역시 서초구 삼성로 (주원최박리)'), ('서동현', '인천광역시 관악구 잠실가 (민석엄김마을)'), ('김광수', '울산광역시 양천구 서초대로'), (... 생략 ...), ('박성현', '전라남도 서산시 가락27길 (준영박문읍)'), ('김성호', '경상남도 영월군 학동거리'), ('백지우', '경기도 계룡시 서초대1로'), ('권유진', '경기도 양주시 서초중앙313가 (춘자나리)'), ('윤서준', '경상남도 청주시 서원 구 서초대64가')]
```

Faker 활용하기

Faker는 앞서 살펴본 name, address 이외에 다른 항목도 제공한다. 대표적인 몇 가지만 알아보자.

항목	설명
fake.name()	이름
fake.address()	주소
fake.postcode()	우편 번호
fake.country()	국가명
fake.company()	회사명
fake.job()	직업명
fake.phone_number()	전화 번호
fake.email()	이메일 주소
fake.user_name()	사용자명
fake.pyint(min_value=0, max_value=100)	0부터 100 사이의 임의의 숫자
fake.ipv4_private()	IP 주소
fake.text()	임의의 문장 (한글 임의의 문장은 `fake.catch_phrase()` 사용)
fake.color_name()	색상명

sympy

sympy는 방정식 기호(symbol)를 사용하게 해 주는 외부 라이브러리이다. 마찬가지로 pip을 이용하여 sympy를 설치하자.

```
C:\> python -m pip install sympy
```

sympy 사용해 보기

시윤이는 가진 돈의 $2/5$로 학용품을 샀다고 한다. 학용품을 사는 데 쓴 돈이 1,760원이라면 남은 돈은 어떻게 구하면 될까?



이 문제는 연습장과 연필만 있으면 쉽게 구할 수 있는 일차방정식 문제이다. 파이썬에서는 sympy를 사용하면 방정식을 쉽게 풀 수 있다. 먼저 다음과 같이 fractions 모듈과 sympy 모듈이 필요하다.

```
>>> from fractions import Fraction
>>> import sympy
```

시윤이가 가진 돈을 x라고 하면 sympy 모듈을 사용하여 다음과 같이 표현할 수 있다.

```
>>> x = sympy.symbols("x")
```

sympy.symbols()는 x처럼 방정식에 사용하는 미지수를 나타내는 기호를 생성할 때 사용한다.

점프 투 파이썬

여러 개의 기호 사용하기

x, y 2개의 미지수가 필요하다면 다음처럼 표현할 수 있다.

```
x, y = sympy.symbols('x y')
```

시윤이가 가진 돈의 $2/5$는 1,760원, 즉 일차방정식 $x * (2/5) = 1760$이므로 이를 코드로 표현하면 다음과 같다.

```
>>> f = sympy.Eq(x*Fraction('2/5'), 1760)
```

`sympy.Eq(a, b)`는 a와 b가 같다는 방정식이다. 여기서 사용한 Fraction은 유리수를 표현할 때 사용하는 표준 라이브러리로, $2/5$를 정확하게 계산하려고 사용했다.

점프 투 파이썬

fractions.Fraction으로 유리수 연산하기

파이썬에서 유리수 연산을 정확하게 하려면 fractions.Fraction을 사용해야 한다.

```
>>> from fractions import Fraction
```

유리수는 다음처럼 Fraction(분자, 분모) 형태로 만들 수 있다.

```
>>> a = Fraction(1, 5)
>>> a
Fraction(1, 5)
```

또는 다음과 같이 Fraction('분자 / 분모')처럼 문자열로 만들 수도 있다.

```
>>> a = Fraction('1/5')
>>> a
Fraction(1, 5)
```

f라는 방정식을 세웠으므로 `sympy.solve(f)` 로 x에 해당하는 값을 구할 수 있다.

```
>>> result = sympy.solve(f)
>>> result
[4400]
```

방정식의 해는 여러 개일 수 있으므로 solve() 함수는 결괏값으로 리스트를 반환한다. 결과를 보면 시윤이가 원래 가진 돈이 4,400원이라는 것을 알 수 있다. 따라서 남은 돈은 다음처럼 가진 돈에서 1,760원을 빼면 된다.

```
>>> remains = result[0] - 1760
>>> remains
2640
```

지금까지 내용을 종합한 풀이는 다음과 같다.

```
# sympy_test.py
from fractions import Fraction
import sympy

# 가지고 있던 돈을 x라고 하자.
x = sympy.symbols("x")

# 가지고 있던 돈의 2/5가 1760원이므로 방정식은 x * (2/5) = 1760 이다.
f = sympy.Eq(x*Fraction('2/5'), 1760)

# 방정식을 만족하는 값(result)을 구한다.
result = sympy.solve(f)  # 결괏값은 리스트

# 남은 돈은 다음과 같이 가지고 있던 돈에서 1760원을 빼면 된다.
remains = result[0] - 1760

print(f'남은 돈은 {remains}원입니다.')
```

프로그램을 실행한 결과는 다음과 같다.

```
남은 돈은 2640원입니다.
```

sympy 활용

$x^2=1$과 같은 2차 방정식의 해를 구해 보자.

```
>>> import sympy
>>> x = sympy.symbols("x")
>>> f = sympy.Eq(x**2, 1)
>>> sympy.solve(f)
[-1, 1]
```

또한 다음과 같은 연립방정식의 해도 구할 수 있다.

```
x + y = 10
x - y = 4
```

```
>>> import sympy
>>> x, y = sympy.symbols('x y')
>>> f1 = sympy.Eq(x+y, 10)
>>> f2 = sympy.Eq(x-y, 4)
>>> sympy.solve([f1, f2])
{x: 7, y: 3}
```

미지수가 2개 이상이면 결괏값이 리스트가 아닌 딕셔너리라는 점에 주의하자.

05장 되새김 문제

죄송합니다. 되새김 문제는 위키독스 웹 사이트에서 비공개 처리되어 있습니다.
되새김 문제에 대한 상세한 내용은 유료 E-book 또는 종이책에서 확인할 수 있습니다.

- 책 구입 안내